

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: DESIGN AND ANALYSIS OF ALGORITHMS

COURSE CODE: CSA06

COURSE OUTCOME COVERED

CO2: Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)

Assessment Tool Weightage: 10%

QUESTIONS: 10

Total Marks:50

Annexure A

DEBUGGING & OPTIMISATION TASK

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the ~~guidelines specified for this assessment.~~

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

DEBUGGING & OPTIMISATION TASK

Q1. Debugging Selection Sort Code

The following Selection Sort code contains logical errors and produces incorrect output for some inputs. Carefully analyze the code, identify all errors in comparison and swapping logic, and explain their root causes. Apply systematic debugging to correct the algorithm step-by-step. Optimize the implementation by reducing unnecessary comparisons if possible. Analyze the corrected algorithm in terms of time complexity. Rewrite the final optimized code with proper structure and comments.

```
def selection_sort(arr):
    n = len(arr)
    for i in range(n):
        min_idx = i
        for j in range(i+1, n):
            if arr[j] > arr[min_idx]: # ERROR
                min_idx = j
        arr[i], arr[min_idx] = arr[min_idx], arr[i]
    return arr
```

Q2. Debugging Linear Search Code

The given Linear Search implementation fails to return correct results when the element is present and sometimes terminates incorrectly. Identify errors in loop conditions and return statements, and explain their causes. Debug the code to ensure correct functionality. Optimize the implementation for better readability and efficiency. Analyze its time complexity and justify the result. Provide a clean and well-documented corrected version.

```
def linear_search(arr, key):
    for i in range(len(arr)):
        if arr[i] == key:
            return -1 # ERROR
    return i # ERROR
```

Q3. Debugging Closest Pair Code

The following Closest Pair brute force code has logical issues in distance calculation and comparison. Identify all errors and explain why incorrect results occur. Debug the code step-by-step to ensure accurate distance computation. Optimize the implementation by avoiding redundant calculations. Analyze the time complexity of the corrected version. Rewrite the final code with proper comments.

```
import math
def closest_pair(points):
    min_dist = 0 # ERROR
    for i in range(len(points)):
        for j in range(i+1, len(points)):
            dist = math.sqrt((points[i][0]-points[j][0])**2 +
```

```

        (points[i][1]-points[j][1])**2)
    if dist > min_dist: # ERROR
        min_dist = dist
    return min_dist

```

Q4. Debugging Convex Hull Logic

The given brute force Convex Hull code incorrectly includes interior points as boundary points. Identify logical errors in orientation checking and explain their impact. Debug the algorithm to correctly identify hull edges. Optimize the implementation by reducing unnecessary checks. Analyze the time complexity of the corrected solution. Present a clean and structured version of the improved code.

```

def orientation(p, q, r):
    return (q[1]-p[1])*(r[0]-q[0]) - (q[0]-p[0])*(r[1]-q[1]) # SIGN ISSUE

```

used incorrectly in hull condition

Q5. Debugging Binary Search Code

The following Binary Search implementation fails for certain inputs due to incorrect boundary updates. Identify all logical errors and explain their causes. Debug the code to handle all edge cases correctly. Optimize the implementation to avoid overflow in mid calculation. Analyze its performance in best, average, and worst cases. Rewrite the corrected code with proper documentation.

```

def binary_search(arr, key):
    low, high = 0, len(arr)-1
    while low < high:
        mid = (low + high) // 2
        if arr[mid] == key:
            return mid
        elif arr[mid] < key:
            high = mid - 1 # ERROR
        else:
            low = mid + 1 # ERROR
    return -1

```

Q6. Debugging Bubble Sort Code

The following Bubble Sort implementation does not correctly sort the array and performs unnecessary iterations. Identify logical errors in loop conditions and swapping mechanism, and explain their causes. Debug the code step-by-step to ensure correct sorting. Optimize the algorithm by reducing unnecessary passes using a flag. Analyze the time complexity of the corrected version. Rewrite the final optimized code with proper comments.

```

def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(n-1):
            if arr[j] < arr[j+1]: # ERROR
                arr[j], arr[j+1] = arr[j+1], arr[j]
    return arr

```

Q7. Debugging Insertion Sort Code

The following Insertion Sort code fails to correctly place elements in sorted order. Identify errors in shifting logic and loop conditions. Explain why incorrect insertion occurs. Debug the algorithm step-by-step to ensure correct functionality. Optimize the code for better readability. Analyze the time complexity. Rewrite the corrected version with proper structure and comments.

```
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j > 0 and arr[j] > key: # ERROR
            arr[j+1] = arr[j]
            j -= 1
        arr[j] = key # ERROR
    return arr
```

Q8. Debugging String Matching Code

The given brute-force string matching code fails to detect all pattern occurrences due to incorrect loop bounds. Identify logical errors in indexing and comparison. Explain why some matches are skipped. Debug the code to correctly find all occurrences. Optimize the implementation to avoid redundant comparisons. Analyze the time complexity of the corrected version. Rewrite the final code with proper comments.

```
def string_match(text, pattern):
    n, m = len(text), len(pattern)
    for i in range(n - m):
        for j in range(m):
            if text[i+j] != pattern[j]:
                break
        if j == m-1:
            print("Match at index", i)
```

Q9. Debugging Maximum Element Search Code

The following code attempts to find the maximum element in an array but returns incorrect results for some inputs. Identify errors in initialization and comparison logic. Explain why incorrect values are produced. Debug the algorithm step-by-step to fix the issue. Optimize the implementation for clarity and correctness. Analyze the time complexity. Provide a clean and corrected version of the code.

```
def find_max(arr):
    max_val = 0 # ERROR
    for i in range(len(arr)):
        if arr[i] < max_val: # ERROR
            max_val = arr[i]
    return max_val
```

Q10. Debugging Pair Sum Code (Brute Force)

The following brute force code is intended to find a pair of elements whose sum equals a target value, but it fails in certain cases. Identify logical errors in

loop structure and return conditions. Explain why valid pairs may not be detected. Debug the code step-by-step to ensure correctness. Optimize the implementation if possible. Analyze the time complexity of the corrected solution. Rewrite the final version with proper documentation.

```
def pair_sum(arr, target):
    for i in range(len(arr)):
        for j in range(len(arr)):
            if i != j and arr[i] + arr[j] == target:
                return (i, j)
    return None
```

Q11. Debugging Selection Sort (Incorrect Swap Position)

The given Selection Sort implementation incorrectly swaps elements even when the minimum index is unchanged, leading to unnecessary operations. Identify the logical error in the swapping mechanism and explain its impact. Debug the code step-by-step to correct the swapping condition. Optimize the implementation to reduce unnecessary swaps. Analyze the time complexity of the corrected version and rewrite the optimized code with proper comments.

```
def selection_sort(arr):
    for i in range(len(arr)):
        min_idx = i
        for j in range(i+1, len(arr)):
            if arr[j] < arr[min_idx]:
                min_idx = j
        arr[i], arr[min_idx] = arr[min_idx], arr[i] # ISSUE
    return arr
```

Q12. Debugging Linear Search (Premature Termination)

The following Linear Search code stops execution prematurely due to incorrect loop control. Identify the logical error causing early termination. Explain why valid elements may not be found. Debug the code to ensure complete traversal. Optimize the implementation for clarity and correctness. Analyze the time complexity and rewrite the corrected code.

```
def linear_search(arr, key):
    for i in range(len(arr)):
        if arr[i] != key:
            return -1 # ERROR
    else:
        return i
```

Q13. Debugging Bubble Sort (Incorrect Loop Bounds)

The given Bubble Sort implementation fails to sort correctly due to improper loop bounds. Identify the error in iteration limits and explain its effect. Debug the code step-by-step to ensure proper comparisons. Optimize by reducing redundant passes. Analyze time complexity and provide a corrected version.

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
```

```

for j in range(n):
    if arr[j] > arr[j+1]: # ERROR
        arr[j], arr[j+1] = arr[j+1], arr[j]

```

Q14. Debugging Binary Search (Infinite Loop Issue)

The Binary Search code below leads to an infinite loop for some inputs. Identify the logical errors in updating search boundaries. Explain why the loop fails to terminate. Debug the algorithm to correctly adjust low and high indices. Optimize mid calculation if necessary. Analyze complexity and rewrite the corrected version.

```

def binary_search(arr, key):
    low, high = 0, len(arr)-1
    while low <= high:
        mid = (low + high)//2
        if arr[mid] == key:
            return mid
        elif arr[mid] < key:
            low = mid # ERROR
        else:
            high = mid # ERROR

```

Q15. Debugging Closest Pair (Wrong Initialization)

The brute-force closest pair algorithm initializes minimum distance incorrectly, leading to wrong outputs. Identify the issue and explain why results are incorrect. Debug the code to correctly initialize and update minimum distance. Optimize to reduce redundant distance calculations. Analyze time complexity and rewrite the corrected code.

```

def closest_pair(points):
    min_dist = 0 # ERROR
    for i in range(len(points)):
        for j in range(i+1, len(points)):
            dist = ((points[i][0]-points[j][0])**2)**0.5
            if dist < min_dist:
                min_dist = dist
    return min_dist

```

Q16. Debugging Convex Hull (Incorrect Orientation Logic)

The given orientation function incorrectly determines turn direction, affecting convex hull formation. Identify the logical error in the formula. Explain its impact on boundary detection. Debug the function to correctly compute orientation. Optimize the overall hull checking process. Analyze time complexity and rewrite the corrected logic.

```

def orientation(p, q, r):
    return (q[0]-p[0])*(r[1]-q[1]) - (q[1]-p[1])*(r[0]-q[0]) # ISSUE

```

Q17. Debugging String Matching (Incorrect Loop Range)

The brute-force string matching code misses valid matches due to incorrect loop range. Identify the error in iteration bounds. Explain why some matches are skipped. Debug the code to ensure all positions are checked. Optimize redundant comparisons. Analyze complexity and rewrite corrected version.

```

for i in range(len(text) - len(pattern)):
    for j in range(len(pattern)):
        if text[i+j] != pattern[j]:
            break

```

Q18. Debugging Maximum Element (Wrong Comparison Direction)

The following code incorrectly finds the maximum element due to reversed comparison logic. Identify the issue and explain why incorrect results occur. Debug the algorithm step-by-step. Optimize initialization and comparison. Analyze time complexity and rewrite corrected code.

```

def find_max(arr):
    max_val = arr[0]
    for i in range(len(arr)):
        if arr[i] < max_val: # ERROR
            max_val = arr[i]
    return max_val

```

Q19. Debugging Pair Sum (Redundant Comparisons)

The brute-force pair sum code checks redundant pairs and increases computation time. Identify inefficiencies in loop structure. Explain why duplicate comparisons occur. Debug to avoid redundant checks. Optimize the algorithm while maintaining correctness. Analyze complexity and rewrite improved version.

```

for i in range(len(arr)):
    for j in range(len(arr)):
        if arr[i] + arr[j] == target:
            return (i, j)

```

Q20. Debugging Insertion Sort (Incorrect Index Placement)

The insertion sort code incorrectly places elements due to wrong index update. Identify the logical error in shifting logic. Explain why elements are misplaced. Debug step-by-step. Optimize code readability. Analyze time complexity and rewrite corrected version.

```

while j >= 0 and arr[j] > key:
    arr[j] = arr[j-1] # ERROR
    j -= 1

```

Q21. Debugging Binary Search Code (Boundary Element Failure)

The following Binary Search implementation fails to correctly detect the target element when it appears at the first or last position of the sorted array. Carefully analyze the boundary update logic and identify why some valid indices are skipped. Apply systematic debugging to correct the loop condition and search updates. Optimize the implementation for clarity and robustness in handling edge cases. Analyze the corrected algorithm in terms of best, average, and worst-case complexity. Rewrite the final optimized code with proper structure and comments.

```

def binary_search(arr, key):
    low, high = 0, len(arr)-1
    while low < high:

```

```

mid = (low + high) // 2
if arr[mid] == key:
    return mid
elif arr[mid] < key:
    low = mid + 1
else:
    high = mid - 1
return -1

```

Q22. Debugging Bubble Sort Code (Missing Early Termination Optimization)

The following Bubble Sort code eventually produces the correct output, but it performs unnecessary passes even when the array becomes sorted early. Carefully analyze the existing implementation and identify the inefficiency in loop execution. Apply debugging and optimization techniques to introduce a flag-based termination mechanism. Explain how this optimization improves the practical performance of the algorithm. Analyze the corrected version in best, average, and worst cases. Rewrite the final optimized code with proper comments and readable structure.

```

def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n-i-1):
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
    return arr

```

Q23. Debugging Linear Search Code (Incorrect Return Outside Logic)

The following Linear Search implementation gives incorrect results because the return statement is misplaced and causes confusion in unsuccessful search cases. Analyze the control flow carefully and identify why the function may produce misleading output. Apply debugging to correct the search logic and ensure proper reporting when the target is found or absent. Optimize the implementation for readability and maintainability. Analyze the time complexity of the corrected algorithm. Rewrite the clean and corrected version with suitable comments.

```

def linear_search(arr, key):
    found = False
    for i in range(len(arr)):
        if arr[i] == key:
            found = True
            pos = i
    return pos

```

Q24. Debugging Selection Sort Code (Wrong Comparison Direction)

The following Selection Sort implementation unintentionally arranges the array in descending order instead of ascending order. Carefully inspect the comparison logic and identify the exact source of the error. Debug the algorithm step-by-step so that it correctly selects the minimum element in each pass. Optimize the implementation by avoiding unnecessary swaps wherever possible. Analyze the time complexity of the corrected algorithm and justify the result. Rewrite the final optimized code with proper documentation.


```
def selection_sort(arr):
    n = len(arr)
    for i in range(n):
        min_idx = i
        for j in range(i+1, n):
            if arr[j] < arr[min_idx]:
                min_idx = j
        arr[i], arr[min_idx] = arr[min_idx], arr[i]
    return arr
```

Q25. Debugging Closest Pair Code (Incomplete Distance Computation)

The following brute force Closest Pair implementation computes distance incorrectly because one coordinate component is ignored. Carefully analyze the formula and identify why the returned minimum distance is inaccurate. Debug the distance calculation step-by-step to include all required terms correctly. Optimize the implementation by minimizing repeated access to point coordinates if possible. Analyze the time complexity of the corrected solution. Rewrite the final code with proper structure and comments.

```
import math
def closest_pair(points):
    min_dist = float('inf')
    for i in range(len(points)):
        for j in range(i+1, len(points)):
            dist = math.sqrt((points[i][0]-points[j][0])**2)
            if dist < min_dist:
                min_dist = dist
    return min_dist
```

Q26. Debugging Convex Hull Logic (Incorrect Inclusion of Interior Points)

The given brute force Convex Hull logic incorrectly accepts some interior edges as hull edges because the validation condition is incomplete. Analyze the hull-checking logic and identify why non-boundary segments are being included. Apply systematic debugging to ensure that only valid outer boundary edges are selected. Optimize the checking process by avoiding unnecessary repeated evaluations where possible. Analyze the time complexity of the corrected brute force solution. Present a clean and structured version of the improved logic.

```
def is_hull_edge(p1, p2, points):
    for p in points:
        val = orientation(p1, p2, p)
        if val < 0:
            return False
    return True
```

Q27. Debugging String Matching Code (Skipped Last Valid Position)

The following brute force string matching implementation fails to examine the last valid starting position of the pattern in the text. Carefully inspect the loop bounds and explain why one potential match is always skipped. Debug the code so that every valid alignment is checked correctly. Optimize the implementation for clarity while

preserving brute force behavior. Analyze the time complexity of the corrected algorithm. Rewrite the final improved code with proper comments.

```
def string_match(text, pattern):
    n, m = len(text), len(pattern)
    matches = []
    for i in range(n - m):
        for j in range(m):
            if text[i+j] != pattern[j]:
                break
        if j == m-1:
            matches.append(i)
    return matches
```

Q28. Debugging Pair Sum Code (Duplicate and Mirror Pair Detection)

The following brute force pair sum implementation may return duplicate or mirror index pairs for the same solution, increasing redundancy in output. Carefully analyze the nested loop structure and identify why repeated pair detection occurs. Debug the algorithm to ensure that each valid pair is checked only once without missing any solution. Optimize the implementation by reducing redundant comparisons while preserving correctness. Analyze the time complexity of the corrected solution. Rewrite the final optimized code with proper comments.

```
def pair_sum_all(arr, target):
    pairs = []
    for i in range(len(arr)):
        for j in range(len(arr)):
            if i != j and arr[i] + arr[j] == target:
                pairs.append((i, j))
    return pairs
```

Q29. Debugging Maximum–Minimum Finder Code (Improper Initialization for Negative Inputs)

The following code attempts to find both minimum and maximum values in an array but fails for all-negative datasets due to incorrect initialization. Carefully identify the root cause of the problem and explain why the output becomes invalid. Debug the implementation step-by-step to correctly handle positive, negative, and mixed inputs. Optimize the logic for clarity and correctness without changing the brute force nature of the solution. Analyze the time complexity of the corrected algorithm. Rewrite the final code with proper documentation.

```
def min_max(arr):
    mn = 0
    mx = 0
    for x in arr:
        if x < mn:
            mn = x
        if x > mx:
            mx = x
    return mn, mx
```

Q30. Debugging Sorting Code (Mixed Ascending and Descending Conditions)

The following sorting implementation contains inconsistent comparison conditions across different passes, causing partially sorted and unstable output. Carefully analyze the logic and identify how conflicting ascending and descending comparisons break correctness. Apply systematic debugging to make the algorithm consistent throughout execution. Optimize the implementation to improve readability and avoid unnecessary operations. Analyze the time complexity of the corrected version and justify the result. Rewrite the final clean and optimized code with suitable comments.

```
def mixed_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n-i-1):
            if i % 2 == 0:
                if arr[j] > arr[j+1]:
                    arr[j], arr[j+1] = arr[j+1], arr[j]
            else:
                if arr[j] < arr[j+1]:
                    arr[j], arr[j+1] = arr[j+1], arr[j]
    return arr
```

Q31. Debugging Selection Sort Code (Incorrect Outer Loop Range)

The following Selection Sort implementation skips the final required pass because of an incorrect outer loop boundary, which may leave the array partially unsorted for certain inputs. Carefully analyze the loop range and explain why the remaining unsorted portion is not handled correctly. Apply systematic debugging to correct the iteration structure and ensure complete sorting. Optimize the implementation by avoiding unnecessary swaps when the minimum element is already in the correct position. Analyze the time complexity of the corrected version and rewrite the final optimized code with proper comments.

```
def selection_sort(arr):
    n = len(arr)
    for i in range(n-2): # ERROR
        min_idx = i
        for j in range(i+1, n):
            if arr[j] < arr[min_idx]:
                min_idx = j
        arr[i], arr[min_idx] = arr[min_idx], arr[i]
    return arr
```

Q32. Debugging Bubble Sort Code (Wrong Inner Loop Limit with Optimization Attempt)

The following Bubble Sort implementation attempts to reduce comparisons in each pass, but the inner loop limit is incorrect and causes some adjacent pairs to remain unchecked. Carefully inspect the loop boundary and explain why certain inversions survive after execution. Debug the algorithm step-by-step to restore correct sorting behavior. Optimize the final version using a swap flag so that unnecessary passes are avoided when the array becomes sorted early. Analyze the corrected algorithm in best, average, and worst cases and rewrite the final code with proper structure.

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n-i-2): # ERROR
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
    return arr
```

Q33. Debugging Linear Search Code (Incorrect Index Variable Update)

The following Linear Search implementation incorrectly updates the result index even after finding the target, causing the function to return the last occurrence instead of the first one. Carefully analyze the logic and identify why the intended search behavior is violated. Apply systematic debugging to ensure that the first valid occurrence is returned immediately when found. Optimize the implementation for readability and correct control flow. Analyze the time complexity of the corrected solution and rewrite the final code with clear comments.

```
def linear_search(arr, key):
    pos = -1
    for i in range(len(arr)):
        if arr[i] == key:
            pos = i
    return pos
```

Q34. Debugging Binary Search Code (Incorrect Mid Recalculation Logic)

The following Binary Search code recalculates the midpoint incorrectly because the boundary updates are inconsistent with the search direction. Carefully inspect the update rules for low and high and explain why valid regions may be discarded. Debug the code step-by-step so that the search interval shrinks correctly in all cases. Optimize the implementation for clarity and proper handling of edge cases. Analyze best, average, and worst-case time complexity and rewrite the corrected code with suitable documentation.

```
def binary_search(arr, key):
    low, high = 0, len(arr)-1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == key:
            return mid
        elif arr[mid] > key:
            low = mid + 1 # ERROR
        else:
            high = mid - 1 # ERROR
    return -1
```

Q35. Debugging Closest Pair Code (Comparison with Wrong Variable)

The following brute force Closest Pair implementation calculates distances correctly but compares the newly computed distance against the wrong reference value. Carefully analyze the update logic and identify why the minimum pair is not always preserved. Apply debugging to ensure that the smallest distance found so far is maintained correctly throughout the nested loops. Optimize the implementation by

avoiding repeated tuple indexing where possible. Analyze the time complexity of the corrected algorithm and rewrite the final version with proper comments.

```
import math
def closest_pair(points):
    min_dist = float('inf')
    for i in range(len(points)):
        for j in range(i+1, len(points)):
            x1, y1 = points[i]
            x2, y2 = points[j]
            dist = math.sqrt((x1-x2)**2 + (y1-y2)**2)
            if dist < min_dist: # ERROR
                min_dist = dist
    return min_dist
```

Q36. Debugging Convex Hull Logic (Collinear Case Mishandling)

The following brute force Convex Hull edge validation logic fails in cases where all remaining points are collinear with a candidate edge. Carefully analyze how collinear points should be handled and explain why the current condition may reject valid hull edges or include incorrect ones. Debug the logic to correctly distinguish between boundary edges and invalid interior segments. Optimize the implementation by reducing repeated orientation checks if possible. Analyze the time complexity of the corrected brute force solution and present the improved code clearly.

```
def is_hull_edge(p1, p2, points):
    for p in points:
        val = orientation(p1, p2, p)
        if val == 0:
            return False # ERROR
        if val < 0:
            return False
    return True
```

Q37. Debugging String Matching Code (Incorrect Match Detection Condition)

The following brute force string matching implementation breaks correctly on mismatches, but the condition used to declare a full match is logically unsafe and depends on the final value of the loop variable. Carefully analyze why this may lead to incorrect behavior in some cases. Apply debugging so that a match is reported only when every character in the pattern has been successfully compared. Optimize the implementation for clarity while retaining brute force logic. Analyze the time complexity and rewrite the final corrected code with comments.

```
def string_match(text, pattern):
    n, m = len(text), len(pattern)
    result = []
    for i in range(n - m + 1):
        for j in range(m):
            if text[i+j] != pattern[j]:
                break
        if j == m-1:
            result.append(i)
```

```
return result
```

Q38. Debugging Pair Sum Code (Premature Return Inside Outer Loop)

The following brute force Pair Sum implementation may terminate too early because the return statement is placed incorrectly inside the outer loop. Carefully inspect the structure and explain why valid pairs later in the array are never considered. Debug the code step-by-step to ensure all necessary combinations are checked before concluding failure. Optimize the nested loops to avoid redundant mirror comparisons while preserving brute force behavior. Analyze the time complexity of the corrected algorithm and rewrite the final version with proper comments.

```
def pair_sum(arr, target):
    for i in range(len(arr)):
        for j in range(i+1, len(arr)):
            if arr[i] + arr[j] == target:
                return (i, j)
    return None # ERROR
```

Q39. Debugging Maximum Element Search Code (Empty List and Initialization Risk)

The following code is intended to find the maximum element, but it fails for empty arrays and uses an unsafe initialization pattern that may lead to errors. Carefully analyze the assumptions made by the code and explain why robustness is affected. Apply systematic debugging to handle edge cases correctly while preserving proper maximum search behavior. Optimize the implementation for clarity and defensive programming. Analyze the time complexity of the corrected solution and rewrite the final code with suitable documentation.

```
def find_max(arr):
    max_val = arr[0] # ISSUE
    for i in range(1, len(arr)):
        if arr[i] > max_val:
            max_val = arr[i]
    return max_val
```

Q40. Debugging Insertion Sort Code (Incorrect Final Placement Index)

The following Insertion Sort implementation performs element shifting, but the final insertion step uses the wrong index and causes misplaced values in the sorted prefix. Carefully analyze the shifting process and identify the exact source of the indexing error. Debug the code step-by-step so that each element is inserted into its correct position after all larger elements are shifted. Optimize the implementation for readability and maintain consistent variable usage. Analyze the corrected algorithm in terms of time complexity and rewrite the final clean version with proper comments.

```
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j >= 0 and arr[j] > key:
            arr[j+1] = arr[j]
            j -= 1
```

```
    arr[j] = key # ERROR
return arr
```

Q41. Debugging Selection Sort Code (Minimum Index Not Reset Properly)

The following Selection Sort implementation fails because the minimum index is initialized only once outside the outer loop, causing comparisons in later passes to use an outdated reference. Carefully analyze why the algorithm loses track of the current unsorted portion. Apply systematic debugging to reset the minimum index correctly in every pass. Optimize the implementation by avoiding unnecessary swaps when the selected minimum is already in place. Analyze the time complexity of the corrected version and rewrite the final optimized code with proper comments.

```
def selection_sort(arr):
    n = len(arr)
    min_idx = 0 # ERROR
    for i in range(n):
        for j in range(i+1, n):
            if arr[j] < arr[min_idx]:
                min_idx = j
        arr[i], arr[min_idx] = arr[min_idx], arr[i]
    return arr
```

Q42. Debugging Bubble Sort Code (Swap Flag Not Reinitialized)

The following Bubble Sort implementation attempts to use a swap flag for optimization, but the flag is not reinitialized at the correct place, causing the algorithm to terminate incorrectly or continue unnecessarily. Carefully inspect the flag control logic and explain its effect on execution. Debug the code step-by-step to ensure proper early termination when the array is already sorted. Optimize the implementation for clarity and practical efficiency. Analyze the corrected algorithm in best, average, and worst cases and rewrite the final code with comments.

```
def bubble_sort(arr):
    n = len(arr)
    swapped = False
    for i in range(n):
        for j in range(0, n-i-1):
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
                swapped = True
        if not swapped:
            break
    return arr
```

Q43. Debugging Linear Search Code (Incorrect Loop Starting Index)

The following Linear Search implementation begins checking from the wrong index and may skip a valid match at the start of the array. Carefully analyze the loop initialization and explain why the first element is ignored. Apply debugging to ensure that every element from the beginning of the array is checked correctly. Optimize the implementation for simplicity and readability while preserving correct functionality. Analyze the time complexity of the corrected version and rewrite the final code with proper comments.

```
def linear_search(arr, key):
    for i in range(1, len(arr)): # ERROR
        if arr[i] == key:
            return i
    return -1
```

Q44. Debugging Binary Search Code (Loop Condition Misses Single Element Case)

The following Binary Search implementation fails when the search interval reduces to a single element because the loop condition is too restrictive. Carefully inspect the termination condition and explain why a valid key may remain unchecked. Debug the algorithm so that all necessary search states are handled correctly, including one-element intervals. Optimize the implementation for clarity and proper edge-case handling. Analyze the corrected algorithm in terms of best, average, and worst-case complexity and rewrite the final version with comments.

```
def binary_search(arr, key):
    low, high = 0, len(arr)-1
    while low < high: # ERROR
        mid = (low + high) // 2
        if arr[mid] == key:
            return mid
        elif arr[mid] < key:
            low = mid + 1
        else:
            high = mid - 1
    return -1
```

Q45. Debugging Closest Pair Code (Redundant Self-Comparison and Wrong Pairing Logic)

The following brute force Closest Pair implementation compares every point with every other point, including itself, which leads to incorrect minimum distance and redundant computation. Carefully analyze the nested loop structure and explain why self-comparison breaks correctness. Debug the algorithm to compare only valid distinct pairs exactly once. Optimize the implementation by reducing duplicate distance evaluations while preserving brute force behavior. Analyze the time complexity of the corrected solution and rewrite the final code with suitable comments.

```
import math
def closest_pair(points):
    min_dist = float('inf')
    for i in range(len(points)):
        for j in range(len(points)):
            dist = math.sqrt((points[i][0]-points[j][0])**2 +
                             (points[i][1]-points[j][1])**2)
            if dist < min_dist:
                min_dist = dist
    return min_dist
```


Q46. Debugging Convex Hull Logic (Candidate Point Not Excluded from Check Properly)

The following brute force Convex Hull helper checks orientation against all points, but it does not handle the candidate edge endpoints correctly, which may interfere with hull validation. Carefully analyze why the endpoints should be excluded from invalidation checks. Apply systematic debugging so that only other points influence the edge decision. Optimize the checking logic to reduce unnecessary operations wherever possible. Analyze the time complexity of the corrected brute force solution and present the improved code clearly.

```
def is_hull_edge(p1, p2, points):
    for p in points:
        val = orientation(p1, p2, p)
        if val < 0:
            return False
    return True
```

Q47. Debugging String Matching Code (Incorrect Handling of Empty Pattern Case)

The following brute force string matching function does not correctly handle the special case where the pattern is empty, which can cause ambiguous or unintended output. Carefully analyze the behavior of the loop structure for this edge case and explain why explicit handling is needed. Debug the code so that empty-pattern input is treated consistently and correctly. Optimize the implementation for robustness and readability without changing the brute force strategy for normal cases. Analyze the time complexity of the corrected version and rewrite the final code with proper documentation.

```
def string_match(text, pattern):
    n, m = len(text), len(pattern)
    matches = []
    for i in range(n - m + 1):
        for j in range(m):
            if text[i+j] != pattern[j]:
                break
        else:
            matches.append(i)
    return matches
```

Q48. Debugging Pair Sum Code (Incorrect Inner Loop Start Causes Missed Pairs)

The following brute force Pair Sum implementation starts the inner loop from an incorrect index, causing some valid combinations to be skipped while others are redundantly checked. Carefully inspect the loop structure and explain why pair coverage becomes incomplete. Debug the algorithm so that every valid pair is considered exactly once without duplication. Optimize the nested loops for clarity and reduced redundancy. Analyze the time complexity of the corrected solution and rewrite the final version with proper comments.

```
def pair_sum(arr, target):
    for i in range(len(arr)):
```

```

        for j in range(i+2, len(arr)): # ERROR
            if arr[i] + arr[j] == target:
                return (i, j)
    return None

```

Q49. Debugging Maximum–Minimum Finder Code (Comparison Branches Reversed)

The following code attempts to find both the minimum and maximum values in a list, but the update conditions are reversed, causing wrong outputs for many inputs. Carefully analyze each comparison branch and explain how the variables are being updated incorrectly. Apply systematic debugging to restore correct min-max tracking across the full array. Optimize the implementation for readability and correctness without changing the brute force nature of the scan. Analyze the time complexity of the corrected solution and rewrite the final code with suitable comments.

```

def min_max(arr):
    mn = arr[0]
    mx = arr[0]
    for x in arr[1:]:
        if x > mn: # ERROR
            mn = x
        if x < mx: # ERROR
            mx = x
    return mn, mx

```

Q50. Debugging Insertion Sort Code (Wrong Direction in Shift Comparison)

The following Insertion Sort implementation uses an incorrect comparison condition in the while loop, causing smaller elements to remain in the wrong position and breaking ascending order. Carefully analyze the shifting logic and explain why the sorted prefix is not maintained correctly. Debug the algorithm step-by-step so that larger elements shift right and the key is inserted in the proper place. Optimize the implementation for clarity and maintain correct indexing throughout. Analyze the corrected algorithm in terms of time complexity and rewrite the final clean code with proper comments.

```

def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j >= 0 and arr[j] < key: # ERROR
            arr[j+1] = arr[j]
            j -= 1
        arr[j+1] = key
    return arr

```

Q51. Debugging Selection Sort Code (Swap Executed Inside Inner Loop)

The following Selection Sort implementation performs swapping inside the inner loop before the minimum element for the pass is fully identified, which causes unstable and incorrect partial ordering. Carefully analyze the execution flow and explain why repeated early swaps break the intended behavior of Selection Sort. Apply systematic debugging to ensure the minimum element is selected completely before

any swap occurs. Optimize the implementation by performing a swap only once per outer pass when necessary. Analyze the time complexity of the corrected version and rewrite the final optimized code with proper comments.

```
def selection_sort(arr):
    n = len(arr)
    for i in range(n):
        min_idx = i
        for j in range(i+1, n):
            if arr[j] < arr[min_idx]:
                min_idx = j
        arr[i], arr[min_idx] = arr[min_idx], arr[i] # ERROR
    return arr
```

Q52. Debugging Bubble Sort Code (Inner Loop Uses Fixed Upper Bound)

The following Bubble Sort implementation always runs the inner loop up to the last index, even though the largest elements have already settled at the end after each pass. Carefully analyze why this does not affect correctness in every case but still introduces inefficiency and can increase the risk of unnecessary comparisons. Debug the code to use the proper shrinking inner loop boundary. Optimize the implementation further using a swap flag to stop early when the array becomes sorted. Analyze the corrected algorithm in best, average, and worst cases and rewrite the final version with comments.

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n-1): # ISSUE
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
    return arr
```

Q53. Debugging Linear Search Code (Wrong Default Return Value for Missing Key)

The following Linear Search function returns 0 when the element is not found, which makes the result indistinguishable from a successful search at the first position. Carefully analyze why this behavior is logically incorrect and how it can mislead the caller. Apply debugging to ensure a proper failure indicator is returned only when the full search completes unsuccessfully. Optimize the implementation for clarity and straightforward control flow. Analyze the time complexity of the corrected solution and rewrite the final code with suitable comments.

```
def linear_search(arr, key):
    for i in range(len(arr)):
        if arr[i] == key:
            return i
    return 0 # ERROR
```

Q54. Debugging Binary Search Code (Middle Element Excluded Incorrectly from Search Space)

The following Binary Search implementation updates the search boundaries in a way

that may exclude valid candidates too aggressively, especially when the target lies adjacent to the midpoint. Carefully inspect the low and high update logic and explain why a correct element may never be checked. Debug the algorithm step-by-step so that the midpoint is handled consistently and the remaining valid region is preserved. Optimize the implementation for robust edge-case handling and readability. Analyze best, average, and worst-case complexity and rewrite the corrected code with proper documentation.

```
def binary_search(arr, key):
    low, high = 0, len(arr)-1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == key:
            return mid
        elif arr[mid] < key:
            low = mid + 2 # ERROR
        else:
            high = mid - 2 # ERROR
    return -1
```

Q55. Debugging Closest Pair Code (Minimum Distance Updated with Pair Index Difference)

The following brute force Closest Pair implementation calculates Euclidean distance correctly, but it updates the minimum value using the index difference instead of the actual distance in one branch of the logic. Carefully analyze why this produces meaningless results for many point configurations. Apply systematic debugging to ensure that only true geometric distance is stored and compared throughout the execution. Optimize the implementation by avoiding redundant tuple access where possible. Analyze the time complexity of the corrected solution and rewrite the final code with proper comments.

```
import math
def closest_pair(points):
    min_dist = float('inf')
    for i in range(len(points)):
        for j in range(i+1, len(points)):
            dist = math.sqrt((points[i][0]-points[j][0])**2 +
                             (points[i][1]-points[j][1])**2)
            if dist < min_dist:
                min_dist = j - i # ERROR
    return min_dist
```

Q56. Debugging Convex Hull Logic (Orientation Sign Tested on Only One Side)

The following brute force Convex Hull helper assumes that every valid hull edge must keep all other points strictly on one chosen side of the line, but it does not properly account for the fact that a valid hull edge may also be formed when all points lie on the opposite side. Carefully analyze why checking only one orientation sign can reject correct boundary edges. Debug the logic so that an edge is accepted when all remaining points stay on either one consistent side or are collinear. Optimize the implementation by structuring the checks clearly and efficiently. Analyze the time

complexity of the corrected brute force solution and present the improved logic with suitable comments.

```
def is_hull_edge(p1, p2, points):
    for p in points:
        val = orientation(p1, p2, p)
        if val > 0: # ISSUE
            return False
    return True
```

Q57. Debugging String Matching Code (Pattern Index Leaks After Mismatch)

The following brute force string matching implementation relies on the loop variable after a mismatch occurs, which may incorrectly label a partial comparison as a complete match. Carefully inspect the control flow and explain why the final value of the inner loop index is not always a safe indicator of success. Apply systematic debugging so that a match is recorded only when all characters of the pattern are verified. Optimize the code for readability and consistent behavior without changing the brute force nature of the approach. Analyze the time complexity of the corrected version and rewrite the final code with proper comments.

```
def string_match(text, pattern):
    n, m = len(text), len(pattern)
    result = []
    for i in range(n - m + 1):
        matched = False
        for j in range(m):
            if text[i+j] != pattern[j]:
                matched = False
                break
        matched = True
        if matched:
            result.append(i)
    return result
```

Q58. Debugging Pair Sum Code (Same Element Allowed Through Incorrect Loop Design)

The following brute force Pair Sum implementation may incorrectly use the same element twice because of the way the inner loop is initialized and the missing restriction on pair formation. Carefully analyze why this leads to invalid solutions when the target equals twice the value of a single element. Debug the nested loops so that only distinct indices are considered and each valid unordered pair is checked exactly once. Optimize the implementation by avoiding redundant mirror comparisons. Analyze the time complexity of the corrected solution and rewrite the final version with proper comments.

```
def pair_sum(arr, target):
    for i in range(len(arr)):
        for j in range(i, len(arr)): # ERROR
            if arr[i] + arr[j] == target:
                return (i, j)
    return None
```

Q59. Debugging Maximum–Minimum Finder Code (Else Branch Prevents Dual Update Logic)

The following min–max finder uses an `if...elif` structure that can hide logical clarity and may create confusion when extending or debugging the code, especially around boundary initialization and unusual input patterns. Carefully analyze how the update logic works and identify why the structure is risky for maintainability and reasoning. Apply systematic debugging to produce a clearer and more reliable implementation that independently checks minimum and maximum values. Optimize the code for readability and robust handling of edge cases such as empty lists. Analyze the time complexity of the corrected solution and rewrite the final code with suitable documentation.

```
def min_max(arr):
    mn = arr[0]
    mx = arr[0]
    for x in arr[1:]:
        if x < mn:
            mn = x
        elif x > mx:
            mx = x
    return mn, mx
```

Q60. Debugging Insertion Sort Code (Key Overwritten Before Insertion Completes)

The following Insertion Sort implementation overwrites the key variable during the shifting process, causing the original value to be lost before it can be inserted into its correct place. Carefully analyze the role of the key and explain why modifying it during shifts breaks the algorithm. Apply debugging step-by-step so that all larger elements are shifted right while the original key is preserved until final placement. Optimize the implementation for clarity, correctness, and consistent indexing. Analyze the corrected algorithm in terms of time complexity and rewrite the final clean code with proper comments.

```
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j >= 0 and arr[j] > key:
            key = arr[j]      # ERROR
            arr[j+1] = arr[j]
            j -= 1
        arr[j+1] = key
    return arr
```

Q61. Debugging Selection Sort Code (Minimum Index Updated with Value Instead of Position)

The following Selection Sort implementation incorrectly stores the minimum element value in `min_idx` instead of its index, which breaks the swapping step and produces invalid behavior for many inputs. Carefully analyze the comparison and update logic to identify why the algorithm confuses values with positions. Apply systematic debugging to ensure that only indices are tracked throughout the selection process.

Optimize the corrected implementation by avoiding unnecessary swaps when the current element is already the smallest. Analyze the time complexity of the corrected version and rewrite the final optimized code with proper comments.

```
def selection_sort(arr):
    n = len(arr)
    for i in range(n):
        min_idx = i
        for j in range(i+1, n):
            if arr[j] < arr[min_idx]:
                min_idx = arr[j] # ERROR
        arr[i], arr[min_idx] = arr[min_idx], arr[i]
    return arr
```

Q62. Debugging Bubble Sort Code (Swap Flag Reset Inside Inner Loop)

The following Bubble Sort implementation tries to use a swap flag for early termination, but the flag is reset inside the inner loop, causing incorrect termination decisions. Carefully inspect the control flow and explain why the optimization becomes logically unsound. Debug the algorithm so that the flag correctly reflects whether any swap occurred during the full pass. Optimize the final implementation for both correctness and practical efficiency. Analyze the corrected version in best, average, and worst cases and rewrite the final code with suitable comments.

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        swapped = False
        for j in range(0, n-i-1):
            swapped = False # ERROR
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
                swapped = True
        if not swapped:
            break
    return arr
```

Q63. Debugging Linear Search Code (Condition Negated Incorrectly)

The following Linear Search implementation uses an incorrect negated condition, which causes it to return the wrong index for the first non-matching element instead of the actual target. Carefully analyze the comparison logic and explain why the search stops incorrectly. Apply systematic debugging to ensure that the algorithm returns the index only when the key is genuinely found. Optimize the corrected implementation for clarity and maintainable control flow. Analyze the time complexity of the corrected solution and rewrite the final code with proper comments.

```
def linear_search(arr, key):
    for i in range(len(arr)):
        if arr[i] != key: # ERROR
            return i
    return -1
```

Q64. Debugging Binary Search Code (Incorrect Midpoint Formula with Bias)

The following Binary Search implementation computes the midpoint using a biased formula that can skip important candidates and distort the search interval for some inputs. Carefully inspect the midpoint calculation and explain why the search space is not divided correctly. Debug the algorithm to use a proper midpoint computation and consistent boundary updates. Optimize the implementation for readability and accurate handling of all edge cases. Analyze the corrected algorithm in best, average, and worst cases and rewrite the final version with proper documentation.

```
def binary_search(arr, key):
    low, high = 0, len(arr)-1
    while low <= high:
        mid = (low + high + 1) // 2 + 1 # ERROR
        if arr[mid] == key:
            return mid
        elif arr[mid] < key:
            low = mid + 1
        else:
            high = mid - 1
    return -1
```

Q65. Debugging Closest Pair Code (Minimum Distance Reinitialized in Outer Loop)

The following brute force Closest Pair implementation incorrectly reinitializes the minimum distance inside the outer loop, causing previously discovered smaller distances to be lost. Carefully analyze why the algorithm fails to preserve the global minimum across all pairs. Apply systematic debugging to ensure that the minimum value is maintained correctly throughout the full execution. Optimize the implementation by reducing repeated tuple indexing where possible. Analyze the time complexity of the corrected solution and rewrite the final code with suitable comments.

```
import math
def closest_pair(points):
    for i in range(len(points)):
        min_dist = float('inf') # ERROR
        for j in range(i+1, len(points)):
            dist = math.sqrt((points[i][0]-points[j][0])**2 +
                             (points[i][1]-points[j][1])**2)
            if dist < min_dist:
                min_dist = dist
    return min_dist
```

Q66. Debugging Convex Hull Logic (Orientation Check Ignores Collinear Boundary Extension)

The following brute force Convex Hull helper accepts collinear points without checking whether they lie beyond the candidate segment, which may lead to incorrect hull edges and inclusion of shorter internal segments. Carefully analyze why simple collinearity alone is not sufficient for validating a boundary edge. Debug the logic to correctly handle collinear cases by considering segment extent and true boundary structure. Optimize the implementation for clearer separation of orientation

testing and segment validation. Analyze the time complexity of the corrected brute force solution and present the improved logic with proper comments.

```
def is_hull_edge(p1, p2, points):
    for p in points:
        val = orientation(p1, p2, p)
        if val < 0:
            return False
    return True # ISSUE: collinear points not checked for segment extent
```

Q67. Debugging String Matching Code (Inner Loop Starts at Wrong Offset)

The following brute force string matching implementation starts comparing the pattern from the wrong character index, which causes the first character of the pattern to be ignored and leads to incorrect matches. Carefully analyze the inner loop structure and explain why partial alignment breaks correctness. Apply systematic debugging so that the comparison begins from the first character at every possible text position. Optimize the implementation for clarity while preserving the brute force strategy. Analyze the time complexity of the corrected version and rewrite the final code with suitable comments.

```
def string_match(text, pattern):
    n, m = len(text), len(pattern)
    matches = []
    for i in range(n - m + 1):
        for j in range(1, m): # ERROR
            if text[i+j] != pattern[j]:
                break
        else:
            matches.append(i)
    return matches
```

Q68. Debugging Pair Sum Code (Wrong Return Value Uses Elements Instead of Indices)

The following brute force Pair Sum implementation returns the element values instead of their indices, even though the task requires reporting positions. Carefully analyze the return statement and explain why this creates inconsistency with the intended function behavior. Debug the algorithm so that it reports the correct pair of indices while still checking each valid combination only once. Optimize the implementation to reduce redundant comparisons and improve readability. Analyze the time complexity of the corrected solution and rewrite the final code with proper comments.

```
def pair_sum(arr, target):
    for i in range(len(arr)):
        for j in range(i+1, len(arr)):
            if arr[i] + arr[j] == target:
                return (arr[i], arr[j]) # ERROR
    return None
```

Q69. Debugging Maximum–Minimum Finder Code (Loop Starts from Wrong Position with Uninitialized Input Check)

The following min–max finder begins scanning from index 0 even after initializing both variables with the first element, which does not always break correctness but introduces redundant comparison and hides the absence of an empty-list check. Carefully analyze the structure and explain why this design is weak in terms of robustness and clarity. Apply systematic debugging to handle empty input safely and remove unnecessary repeated comparison of the first element. Optimize the implementation for maintainability and defensive programming. Analyze the time complexity of the corrected solution and rewrite the final code with suitable documentation.

```
def min_max(arr):
    mn = arr[0]
    mx = arr[0]
    for i in range(len(arr)): # ISSUE
        if arr[i] < mn:
            mn = arr[i]
        if arr[i] > mx:
            mx = arr[i]
    return mn, mx
```

Q70. Debugging Insertion Sort Code (Shifting Stops One Step Too Early)

The following Insertion Sort implementation stops the shifting process too early because the loop condition excludes index 0 from comparison, causing incorrect placement when the key belongs at the beginning of the array. Carefully analyze the while-loop boundary and explain why the first position is not handled correctly. Apply debugging step-by-step so that all larger elements, including the element at index 0, are shifted when necessary. Optimize the implementation for correctness, clarity, and consistent indexing. Analyze the corrected algorithm in terms of time complexity and rewrite the final clean code with proper comments.

```
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j > 0 and arr[j] > key: # ERROR
            arr[j+1] = arr[j]
            j -= 1
        arr[j+1] = key
    return arr
```

Q71. Debugging Selection Sort Code (Inner Loop Starts from Wrong Position)

The following Selection Sort implementation starts the inner loop from index 0 instead of the current unsorted position, causing already sorted elements to be reconsidered and possibly disturbed. Carefully analyze why the algorithm fails to preserve the sorted prefix after each pass. Apply systematic debugging to ensure that only the unsorted portion is scanned to find the minimum element. Optimize the implementation by avoiding unnecessary swaps when the selected minimum is already in place. Analyze the time complexity of the corrected version and rewrite the final optimized code with proper comments.

```
def selection_sort(arr):
```

```

n = len(arr)
for i in range(n):
    min_idx = i
    for j in range(0, n): # ERROR
        if arr[j] < arr[min_idx]:
            min_idx = j
    arr[i], arr[min_idx] = arr[min_idx], arr[i]
return arr

```

Q72. Debugging Bubble Sort Code (Adjacent Comparison Uses Wrong Index Pair)

The following Bubble Sort implementation compares non-adjacent elements due to an incorrect index reference inside the inner loop, which breaks the fundamental logic of repeated neighboring swaps. Carefully inspect the comparison and swapping statements and explain why local inversions are not resolved correctly. Debug the code step-by-step so that only adjacent elements are compared and exchanged. Optimize the corrected version using a swap flag to reduce unnecessary passes. Analyze the algorithm in best, average, and worst cases and rewrite the final code with proper comments.

```

def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n-i-1):
            if arr[j] > arr[i]: # ERROR
                arr[j], arr[i] = arr[i], arr[j]
    return arr

```

Q73. Debugging Linear Search Code (Search Stops After First Iteration)

The following Linear Search implementation mistakenly places the failure return statement inside the loop, causing the algorithm to stop after checking only the first element. Carefully analyze the control flow and explain why valid matches later in the array are never reached. Apply systematic debugging to ensure the full array is scanned before concluding that the key is absent. Optimize the implementation for clarity and correct control structure. Analyze the time complexity of the corrected solution and rewrite the final code with suitable comments.

```

def linear_search(arr, key):
    for i in range(len(arr)):
        if arr[i] == key:
            return i
        else:
            return -1 # ERROR

```

Q74. Debugging Binary Search Code (Sorted Order Assumption Violated by Reverse Comparison Logic)

The following Binary Search implementation uses reversed comparison updates, making it behave as though the array were sorted in descending order even though the input is assumed to be ascending. Carefully inspect the decision logic and explain why the valid search interval moves in the wrong direction. Debug the algorithm so that boundary updates correctly reflect ascending-order data. Optimize

the implementation for readability and robust edge-case handling. Analyze the corrected version in best, average, and worst cases and rewrite the final code with proper documentation.

```
def binary_search(arr, key):
    low, high = 0, len(arr)-1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == key:
            return mid
        elif key < arr[mid]:
            low = mid + 1 # ERROR
        else:
            high = mid - 1 # ERROR
    return -1
```

Q75. Debugging Closest Pair Code (Distance Formula Uses Addition Instead of Difference)

The following brute force Closest Pair implementation uses an incorrect distance formula by adding coordinates instead of taking their differences, which produces meaningless geometric distances. Carefully analyze the mathematical expression and explain why the computed minimum is invalid. Apply systematic debugging to restore the correct Euclidean distance formula for all point pairs. Optimize the implementation by storing coordinates in temporary variables for clarity. Analyze the time complexity of the corrected solution and rewrite the final code with proper comments.

```
import math
def closest_pair(points):
    min_dist = float('inf')
    for i in range(len(points)):
        for j in range(i+1, len(points)):
            dist = math.sqrt((points[i][0] + points[j][0])**2 +
                             (points[i][1] + points[j][1])**2) # ERROR
            if dist < min_dist:
                min_dist = dist
    return min_dist
```

Q76. Debugging Convex Hull Logic (Endpoint Reversal Not Treated Consistently)

The following brute force Convex Hull helper may produce inconsistent decisions for the same edge when the two endpoints are supplied in reversed order. Carefully analyze how orientation signs change when the segment direction is reversed and explain why the current logic can reject a valid hull edge in one direction but accept it in the other. Debug the code so that edge validation works consistently regardless of endpoint order. Optimize the implementation by structuring the side-checking logic more clearly. Analyze the time complexity of the corrected brute force solution and present the improved logic with proper comments.

```
def is_hull_edge(p1, p2, points):
    for p in points:
```

```

val = orientation(p1, p2, p)
if val < 0:
    return False
return True

```

Q77. Debugging String Matching Code (Outer Loop Overruns Valid Alignment Range)

The following brute force string matching implementation allows the outer loop to continue beyond the last valid alignment of the pattern in the text, which can lead to indexing errors or invalid comparisons. Carefully inspect the loop bound and explain why the text window must stop at a precise position. Apply systematic debugging to ensure that only valid starting indices are checked. Optimize the implementation for readability and safe control flow while preserving brute force logic. Analyze the time complexity of the corrected version and rewrite the final code with suitable comments.

```

def string_match(text, pattern):
    n, m = len(text), len(pattern)
    result = []
    for i in range(n): # ERROR
        for j in range(m):
            if text[i+j] != pattern[j]:
                break
        else:
            result.append(i)
    return result

```

Q78. Debugging Pair Sum Code (Loop Structure Misses Final Valid Pair)

The following brute force Pair Sum implementation stops the inner loop too early, which causes the last possible pair involving the final element to be skipped. Carefully analyze the loop bound and explain why the complete set of unordered pairs is not explored. Debug the nested loops so that every valid pair is checked exactly once. Optimize the implementation by preserving distinct-index checking without redundant mirror comparisons. Analyze the time complexity of the corrected solution and rewrite the final code with proper comments.

```

def pair_sum(arr, target):
    for i in range(len(arr)):
        for j in range(i+1, len(arr)-1): # ERROR
            if arr[i] + arr[j] == target:
                return (i, j)
    return None

```

Q79. Debugging Maximum–Minimum Finder Code (Minimum and Maximum Initialized to Opposite Extremes Incorrectly)

The following min–max finder initializes the minimum and maximum variables in a reversed and unsafe manner, which produces incorrect results for many arrays and obscures the intended logic. Carefully analyze the initialization step and explain why the tracking variables no longer represent valid starting extremes. Apply systematic debugging so that both minimum and maximum are initialized correctly and updated consistently throughout the scan. Optimize the implementation for robustness and

clarity, including safe handling of invalid input. Analyze the time complexity of the corrected solution and rewrite the final code with suitable documentation.

```
def min_max(arr):
    mn = float('-inf') # ERROR
    mx = float('inf') # ERROR
    for x in arr:
        if x < mn:
            mn = x
        if x > mx:
            mx = x
    return mn, mx
```

Q80. Debugging Insertion Sort Code (Key Inserted Before All Shifts Complete)

The following Insertion Sort implementation inserts the key value inside the shifting loop, causing premature placement and breaking the sorted prefix invariant. Carefully analyze the order of operations and explain why the key must be inserted only after all larger elements have been shifted right. Apply debugging step-by-step so that the shifting and insertion phases are clearly separated. Optimize the implementation for readability and correct indexing throughout. Analyze the corrected algorithm in terms of time complexity and rewrite the final clean code with proper comments.

```
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j >= 0 and arr[j] > key:
            arr[j+1] = key # ERROR
            arr[j] = arr[j]
            j -= 1
    return arr
```

Q81. Debugging Selection Sort Code (Minimum Candidate Compared Against Wrong Position)

The following Selection Sort implementation compares each new element against `arr[i]` instead of the current minimum candidate, which causes the algorithm to ignore smaller values found earlier in the same pass. Carefully analyze why the minimum element is not tracked correctly across the inner loop. Apply systematic debugging so that every comparison is made against the current minimum index. Optimize the implementation by skipping swaps when the selected minimum is already in its correct position. Analyze the time complexity of the corrected version and rewrite the final optimized code with proper comments.

```
def selection_sort(arr):
    n = len(arr)
    for i in range(n):
        min_idx = i
        for j in range(i+1, n):
            if arr[j] < arr[i]: # ERROR
                min_idx = j
        arr[i], arr[min_idx] = arr[min_idx], arr[i]
```

```
return arr
```

Q82. Debugging Bubble Sort Code (Outer Loop Starts from Wrong Pass Index)

The following Bubble Sort implementation starts its outer loop from index **1**, which skips the first full pass and may leave the largest element unsafely placed in the unsorted region. Carefully inspect the loop structure and explain why the full number of passes is no longer guaranteed. Apply debugging to restore the correct pass sequence from the beginning. Optimize the final code using a swap flag so that early termination is possible when the array becomes sorted. Analyze the corrected algorithm in best, average, and worst cases and rewrite the final version with proper comments.

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(1, n): # ERROR
        for j in range(0, n-i):
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
    return arr
```

Q83. Debugging Linear Search Code (Search Continues but Result Is Overwritten Incorrectly)

The following Linear Search implementation tries to record the location of the key, but it overwrites the result even after a successful match because of an incorrect **else** branch. Carefully analyze the control flow and explain why a later non-matching element can erase a previously found answer. Apply systematic debugging so that the result remains valid once the key is found. Optimize the implementation for simplicity and immediate termination on success. Analyze the time complexity of the corrected solution and rewrite the final code with suitable comments.

```
def linear_search(arr, key):
    pos = -1
    for i in range(len(arr)):
        if arr[i] == key:
            pos = i
        else:
            pos = -1 # ERROR
    return pos
```

Q84. Debugging Binary Search Code (Loop Updates Skip Midpoint Neighbors)

The following Binary Search implementation incorrectly adjusts the boundaries by more than one position, causing valid neighboring candidates around the midpoint to be skipped. Carefully inspect the update rules and explain why the search interval shrinks too aggressively. Apply debugging so that the remaining candidate range is reduced correctly without losing possible matches. Optimize the implementation for readability and correct edge-case handling. Analyze best, average, and worst-case complexity and rewrite the corrected code with proper documentation.

```
def binary_search(arr, key):
    low, high = 0, len(arr)-1
    while low <= high:
```

```

mid = (low + high) // 2
if arr[mid] == key:
    return mid
elif arr[mid] < key:
    low = mid + 2 # ERROR
else:
    high = mid - 2 # ERROR
return -1

```

Q85. Debugging Closest Pair Code (Distance Compared Before Being Fully Computed)

The following brute force Closest Pair implementation compares a partially computed distance value before both coordinate contributions are included, which leads to incorrect minimum selection. Carefully analyze the formula construction and explain why the geometric distance must be completed before comparison. Apply systematic debugging to compute the full Euclidean distance for each valid pair before updating the minimum. Optimize the implementation by storing coordinate components in temporary variables for clarity. Analyze the time complexity of the corrected solution and rewrite the final code with proper comments.

```

import math
def closest_pair(points):
    min_dist = float('inf')
    for i in range(len(points)):
        for j in range(i+1, len(points)):
            dx = (points[i][0] - points[j][0])**2
            if dx < min_dist: # ERROR
                dist = math.sqrt(dx + (points[i][1] - points[j][1])**2)
                min_dist = dist
    return min_dist

```

Q86. Debugging Convex Hull Logic (Mixed-Side Points Not Tracked Correctly)

The following brute force Convex Hull helper does not correctly distinguish whether points lie on both sides of the candidate line segment, so some invalid interior segments may be accepted as hull edges. Carefully analyze why a valid hull edge must keep all other points on only one side or collinear. Apply systematic debugging to track the existence of positive and negative orientations separately. Optimize the logic for clarity and reduced repeated checks wherever possible. Analyze the time complexity of the corrected brute force solution and present the improved code with proper comments.

```

def is_hull_edge(p1, p2, points):
    pos_side = neg_side = False
    for p in points:
        val = orientation(p1, p2, p)
        if val > 0:
            pos_side = True
        if val > 0: # ERROR
            neg_side = True
    return not (pos_side and neg_side)

```


Q87. Debugging String Matching Code (Mismatch Handling Appends Invalid Position)

The following brute force string matching implementation incorrectly appends a match position even when a mismatch occurs at an intermediate character, because the append statement is placed outside the proper success condition. Carefully inspect the control flow and explain why partial matches are being reported as complete ones. Apply debugging so that a position is added only when every character in the pattern matches successfully. Optimize the implementation for clear success detection while preserving brute force behavior. Analyze the time complexity of the corrected version and rewrite the final code with suitable comments.

```
def string_match(text, pattern):
    n, m = len(text), len(pattern)
    result = []
    for i in range(n - m + 1):
        for j in range(m):
            if text[i+j] != pattern[j]:
                break
        result.append(i) # ERROR
    return result
```

Q88. Debugging Pair Sum Code (Loop Condition Prevents Last Outer Index from Participating Correctly)

The following brute force Pair Sum implementation stops the outer loop one step too early, which may prevent the final useful starting index from participating in valid pair checks. Carefully analyze the loop bound and explain why the complete set of unordered pairs is not always explored. Apply systematic debugging so that every valid pair is considered exactly once without redundant mirror comparisons. Optimize the implementation for correctness and readability. Analyze the time complexity of the corrected solution and rewrite the final code with proper comments.

```
def pair_sum(arr, target):
    for i in range(len(arr)-2): # ERROR
        for j in range(i+1, len(arr)):
            if arr[i] + arr[j] == target:
                return (i, j)
    return None
```

Q89. Debugging Maximum–Minimum Finder Code (Updates Tied to Wrong Variable State)

The following min–max finder updates one tracking variable based on the state of the other, which creates incorrect behavior when the array contains mixed positive and negative values or values appearing in irregular order. Carefully analyze the comparison logic and explain why minimum and maximum must be maintained independently. Apply debugging to ensure that both extremes are tracked correctly throughout the scan. Optimize the implementation for robustness, clarity, and safe input handling. Analyze the time complexity of the corrected solution and rewrite the final code with suitable documentation.

```
def min_max(arr):
    mn = arr[0]
```

```

mx = arr[0]
for x in arr[1:]:
    if x < mx: # ERROR
        mn = x
    if x > mn: # ERROR
        mx = x
return mn, mx

```

Q90. Debugging Insertion Sort Code (Wrong Element Shifted During Inner Loop)

The following Insertion Sort implementation shifts the wrong element during the inner loop, causing values to be duplicated and the original ordering information to be lost. Carefully analyze the shifting step and explain why each larger element must move one position to the right before the key is inserted. Apply systematic debugging so that the correct array element is shifted at every iteration. Optimize the implementation for readability and consistent indexing throughout the algorithm. Analyze the corrected version in terms of time complexity and rewrite the final clean code with proper comments.

```

def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j >= 0 and arr[j] > key:
            arr[j+1] = arr[j+1] # ERROR
            j -= 1
        arr[j+1] = key
    return arr

```

Q91. Debugging Selection Sort Code (Minimum Index Updated with Element Value in C)

The following Selection Sort implementation in C incorrectly stores the minimum element value in the variable meant for the minimum index, which causes invalid swapping and unexpected output for many arrays. Carefully analyze the comparison and update logic to identify why index tracking fails during each pass. Apply systematic debugging so that only positions are stored in the minimum index variable throughout the algorithm. Optimize the corrected implementation by avoiding unnecessary swaps when the current element is already the smallest. Analyze the time complexity of the corrected version and rewrite the final optimized C code with proper comments.

```

void selectionSort(int arr[], int n) {
    int i, j, min_idx, temp;
    for (i = 0; i < n; i++) {
        min_idx = i;
        for (j = i + 1; j < n; j++) {
            if (arr[j] < arr[min_idx]) {
                min_idx = arr[j]; // ERROR
            }
        }
        temp = arr[i];

```

```

        arr[i] = arr[min_idx];
        arr[min_idx] = temp;
    }
}

```

Q92. Debugging Bubble Sort Code (Incorrect Inner Loop Bound in C)

The following Bubble Sort implementation in C uses an incorrect inner loop boundary, which causes out-of-range access or leaves some adjacent pairs unchecked depending on input size. Carefully inspect the loop condition and explain why the algorithm does not safely compare valid neighboring elements in every pass. Debug the code step-by-step so that each pass compares only valid adjacent elements within the unsorted region. Optimize the corrected version using a swap flag to terminate early when the array becomes sorted. Analyze the algorithm in best, average, and worst cases and rewrite the final C program with proper comments.

```

void bubbleSort(int arr[], int n) {
    int i, j, temp;
    for (i = 0; i < n; i++) {
        for (j = 0; j <= n - i - 1; j++) { // ERROR
            if (arr[j] > arr[j + 1]) {
                temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

```

Q93. Debugging Linear Search Code (Premature Failure Return in C)

The following Linear Search implementation in C returns failure from inside the loop immediately after the first non-matching comparison, which prevents the rest of the array from being searched. Carefully analyze the control flow and explain why valid matches later in the array are never reached. Apply systematic debugging so that the search continues until the key is found or the full array has been checked. Optimize the corrected implementation for clarity and direct readability. Analyze the time complexity of the corrected solution and rewrite the final C code with suitable comments.

```

int linearSearch(int arr[], int n, int key) {
    int i;
    for (i = 0; i < n; i++) {
        if (arr[i] == key)
            return i;
        else
            return -1; // ERROR
    }
    return -1;
}

```

Q94. Debugging Binary Search Code (Wrong Boundary Updates in C)

The following Binary Search implementation in C updates the lower and upper

bounds in the wrong direction, making it discard the valid half of the sorted array during search. Carefully inspect the comparison logic and explain why the search interval moves away from the target instead of toward it. Debug the code step-by-step so that the midpoint decisions correctly preserve the remaining candidate region. Optimize the corrected implementation for clarity and proper edge-case handling. Analyze the best, average, and worst-case complexity and rewrite the final C code with proper documentation.

```
int binarySearch(int arr[], int n, int key) {
    int low = 0, high = n - 1, mid;
    while (low <= high) {
        mid = (low + high) / 2;
        if (arr[mid] == key)
            return mid;
        else if (arr[mid] < key)
            high = mid - 1; // ERROR
        else
            low = mid + 1; // ERROR
    }
    return -1;
}
```

Q95. Debugging Closest Pair Code (Wrong Initialization and Comparison in C)

The following brute force Closest Pair implementation in C initializes the minimum distance incorrectly and uses the wrong comparison direction, leading to incorrect results even for simple point sets. Carefully analyze why the algorithm never preserves the actual smallest distance among all pairs. Apply systematic debugging to correctly initialize the minimum distance and update it only when a smaller valid distance is found. Optimize the implementation by avoiding redundant distance calculations where possible. Analyze the time complexity of the corrected solution and rewrite the final C code with proper comments.

```
#include <math.h>
double closestPair(int x[], int y[], int n) {
    int i, j;
    double minDist = 0; // ERROR
    for (i = 0; i < n; i++) {
        for (j = i + 1; j < n; j++) {
            double dist = sqrt((x[i] - x[j]) * (x[i] - x[j]) +
                               (y[i] - y[j]) * (y[i] - y[j]));
            if (dist > minDist) { // ERROR
                minDist = dist;
            }
        }
    }
    return minDist;
}
```

Q96. Debugging Convex Hull Orientation Logic (Sign Error in C)

The following orientation function used in a brute force Convex Hull implementation in C contains a sign-related logical issue that can reverse turn direction and cause

interior edges to be treated as hull edges. Carefully analyze the formula and explain how incorrect orientation affects boundary detection. Debug the function so that clockwise, counterclockwise, and collinear cases are interpreted correctly in the hull logic. Optimize the overall reasoning by structuring the edge-validation check more clearly. Analyze the time complexity of the corrected brute force approach and rewrite the improved C code with suitable comments.

```
int orientation(int px, int py, int qx, int qy, int rx, int ry) {
    return (qy - py) * (rx - qx) - (qx - px) * (ry - qy); // ISSUE
}
```

Q97. Debugging String Matching Code (Last Valid Alignment Skipped in C)

The following brute force string matching implementation in C skips the last valid starting position of the pattern in the text because of an incorrect outer loop bound. Carefully inspect the loop range and explain why one potential match is never examined. Apply systematic debugging so that every valid alignment from the first to the last possible position is checked correctly. Optimize the corrected implementation for readability while preserving the brute force method. Analyze the time complexity of the corrected version and rewrite the final C code with proper comments.

```
#include <stdio.h>
#include <string.h>
void stringMatch(char text[], char pattern[]) {
    int n = strlen(text), m = strlen(pattern);
    int i, j;
    for (i = 0; i < n - m; i++) { // ERROR
        for (j = 0; j < m; j++) {
            if (text[i + j] != pattern[j])
                break;
        }
        if (j == m)
            printf("Match at %d\n", i);
    }
}
```

Q98. Debugging Pair Sum Code (Redundant and Invalid Pair Checks in C)

The following brute force Pair Sum implementation in C checks all ordered pairs, including mirror pairs and self-pairs, which leads to redundant computation and may produce invalid solutions. Carefully analyze the nested loop structure and explain why each valid unordered pair should be considered only once using distinct indices. Debug the code so that redundant and invalid comparisons are eliminated without missing any correct answer. Optimize the implementation for cleaner structure and reduced unnecessary checks. Analyze the time complexity of the corrected solution and rewrite the final C code with proper comments.

```
int pairSum(int arr[], int n, int target, int *a, int *b) {
    int i, j;
    for (i = 0; i < n; i++) {
        for (j = 0; j < n; j++) { // ISSUE
            if (arr[i] + arr[j] == target) {
                *a = i;
```

```

        *b = j;
        return 1;
    }
}
}
return 0;
}

```

Q99. Debugging Maximum Element Search Code (Unsafe Initialization in C)

The following C function attempts to find the maximum element in an array, but it initializes the maximum value to zero, which causes incorrect results when the array contains all negative values. Carefully analyze the initialization step and explain why it breaks correctness for certain input categories. Apply systematic debugging so that the maximum is initialized safely from the data itself and the code handles invalid input appropriately. Optimize the corrected implementation for robustness and clarity. Analyze the time complexity of the corrected solution and rewrite the final C code with suitable comments.

```

int findMax(int arr[], int n) {
    int i, maxVal = 0; // ERROR
    for (i = 0; i < n; i++) {
        if (arr[i] > maxVal)
            maxVal = arr[i];
    }
    return maxVal;
}

```

Q100. Debugging Insertion Sort Code (Incorrect Final Placement Index in C)

The following Insertion Sort implementation in C shifts larger elements correctly, but it places the key back into the wrong array index after the shifting loop ends, which causes misplaced elements and breaks ascending order. Carefully analyze the role of `j` after the loop terminates and explain why the final insertion must occur at the next valid position. Debug the code step-by-step so that the key is inserted exactly where the sorted prefix requires it. Optimize the corrected implementation for readability and consistent indexing. Analyze the corrected algorithm in terms of time complexity and rewrite the final clean C code with proper comments.

```

void insertionSort(int arr[], int n) {
    int i, j, key;
    for (i = 1; i < n; i++) {
        key = arr[i];
        j = i - 1;
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j] = key; // ERROR
    }
}

```

Q101. Debugging Selection Sort Code (Incorrect Swap Condition Causes Partial Sorting in C)

The following Selection Sort implementation in C performs swapping only when the current element is greater than the selected minimum, but the condition is placed incorrectly and may skip necessary exchanges in some cases. Carefully analyze how the swap decision is tied to the final minimum index chosen in each pass. Apply systematic debugging so that the minimum element of the unsorted portion is always placed correctly at the current position. Optimize the corrected implementation by avoiding redundant swaps when the minimum is already in the correct place. Analyze the time complexity of the corrected version and rewrite the final optimized C code with proper comments.

```
void selectionSort(int arr[], int n) {
    int i, j, min_idx, temp;
    for (i = 0; i < n - 1; i++) {
        min_idx = i;
        for (j = i + 1; j < n; j++) {
            if (arr[j] < arr[min_idx]) {
                min_idx = j;
            }
        }
        if (arr[i] < arr[min_idx]) { // ERROR
            temp = arr[i];
            arr[i] = arr[min_idx];
            arr[min_idx] = temp;
        }
    }
}
```

Q102. Debugging Bubble Sort Code (Early Exit Check Placed Inside Inner Loop in C)

The following Bubble Sort implementation in C attempts to terminate early using a swap flag, but the early-exit check is placed inside the inner loop, causing premature termination before the full pass is completed. Carefully inspect the flag logic and explain why one full pass must finish before deciding whether the array is already sorted. Debug the code step-by-step so that early termination happens only after a complete pass with no swaps. Optimize the corrected implementation for both correctness and practical efficiency. Analyze the best, average, and worst-case complexity and rewrite the final C code with proper comments.

```
void bubbleSort(int arr[], int n) {
    int i, j, temp, swapped;
    for (i = 0; i < n - 1; i++) {
        swapped = 0;
        for (j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
                swapped = 1;
            }
        }
    }
}
```

```

    }
    if (!swapped) // ERROR
        break;
}
}
}

```

Q103. Debugging Linear Search Code (Returned Position Shifted by One in C)

The following Linear Search implementation in C returns the position as $i + 1$ instead of the actual array index, which creates inconsistent results for zero-based indexing. Carefully analyze the return logic and explain why this causes incorrect interpretation of the found position by the caller. Apply systematic debugging so that the algorithm reports the exact index of the key in the array. Optimize the corrected implementation for clarity and maintain consistent indexing conventions. Analyze the time complexity of the corrected solution and rewrite the final C code with suitable comments.

```

int linearSearch(int arr[], int n, int key) {
    int i;
    for (i = 0; i < n; i++) {
        if (arr[i] == key)
            return i + 1; // ERROR
    }
    return -1;
}

```

Q104. Debugging Binary Search Code (Single-Element Interval Not Checked Properly in C)

The following Binary Search implementation in C uses a loop condition that fails to evaluate the final single-element interval, which may cause a valid key to remain unchecked. Carefully inspect the termination condition and explain why the last candidate must still be processed before declaring failure. Debug the algorithm so that every valid search interval, including one-element ranges, is handled correctly. Optimize the corrected implementation for clear structure and reliable edge-case behavior. Analyze the best, average, and worst-case complexity and rewrite the final C code with proper documentation.

```

int binarySearch(int arr[], int n, int key) {
    int low = 0, high = n - 1, mid;
    while (low < high) { // ERROR
        mid = (low + high) / 2;
        if (arr[mid] == key)
            return mid;
        else if (arr[mid] < key)
            low = mid + 1;
        else
            high = mid - 1;
    }
    return -1;
}

```


Q105. Debugging Closest Pair Code (Duplicate Minimum Reset Inside Inner Logic in C)

The following brute force Closest Pair implementation in C resets the minimum distance incorrectly whenever a new outer-loop iteration begins, which destroys previously found smaller distances. Carefully analyze the scope of the minimum-distance variable and explain why it must persist across all pair comparisons. Apply systematic debugging so that the smallest distance among all valid pairs is preserved throughout the computation. Optimize the implementation by reducing repeated coordinate access and improving readability. Analyze the time complexity of the corrected solution and rewrite the final C code with proper comments.

```
#include <math.h>
double closestPair(int x[], int y[], int n) {
    int i, j;
    double minDist = 1e9;
    for (i = 0; i < n; i++) {
        minDist = 1e9; // ERROR
        for (j = i + 1; j < n; j++) {
            double dist = sqrt((x[i] - x[j]) * (x[i] - x[j]) +
                               (y[i] - y[j]) * (y[i] - y[j]));
            if (dist < minDist)
                minDist = dist;
        }
    }
    return minDist;
}
```

Q106. Debugging Convex Hull Orientation Usage (Collinear Points Mishandled in C)

The following Convex Hull helper in C treats every collinear point as invalid for a hull edge, which may incorrectly reject true boundary segments lying along the outer edge of the point set. Carefully analyze how collinear points should be interpreted in brute force hull validation. Debug the logic so that collinear cases are handled correctly without automatically discarding valid boundary edges. Optimize the corrected implementation for clearer geometric reasoning and reduced unnecessary checks. Analyze the time complexity of the corrected brute force solution and rewrite the final C logic with suitable comments.

```
int isHullEdge(int p1x, int p1y, int p2x, int p2y, int x[], int y[], int n) {
    int i;
    for (i = 0; i < n; i++) {
        int val = (p2y - p1y) * (x[i] - p2x) - (p2x - p1x) * (y[i] - p2y);
        if (val <= 0) // ERROR
            return 0;
    }
    return 1;
}
```

Q107. Debugging String Matching Code (Match Reporting on Partial Success in C)

The following brute force string matching implementation in C incorrectly prints a

match when the last compared character happens to agree, even if an earlier mismatch already occurred in the same alignment. Carefully inspect the control flow and explain why full-pattern success must be distinguished from partial agreement. Apply systematic debugging so that a match is reported only when every character of the pattern matches consecutively. Optimize the corrected implementation for clear success detection while preserving brute force behavior. Analyze the time complexity of the corrected version and rewrite the final C code with proper comments.

```
#include <stdio.h>
#include <string.h>
void stringMatch(char text[], char pattern[]) {
    int n = strlen(text), m = strlen(pattern);
    int i, j;
    for (i = 0; i <= n - m; i++) {
        for (j = 0; j < m; j++) {
            if (text[i + j] != pattern[j])
                break;
        }
        if (text[i + j - 1] == pattern[j - 1]) // ERROR
            printf("Match at %d\n", i);
    }
}
```

Q108. Debugging Pair Sum Code (Failure Due to Incorrect Inner Loop Limit in C)

The following brute force Pair Sum implementation in C stops the inner loop one step too early, which may miss a valid pair involving the final array element. Carefully analyze the nested loop bounds and explain why the complete set of unordered pairs must be explored exactly once. Debug the algorithm so that every valid distinct pair is checked without redundant mirror comparisons. Optimize the corrected implementation for readability and reduced unnecessary work. Analyze the time complexity of the corrected solution and rewrite the final C code with proper comments.

```
int pairSum(int arr[], int n, int target, int *a, int *b) {
    int i, j;
    for (i = 0; i < n - 1; i++) {
        for (j = i + 1; j < n - 1; j++) { // ERROR
            if (arr[i] + arr[j] == target) {
                *a = i;
                *b = j;
                return 1;
            }
        }
    }
    return 0;
}
```

Q109. Debugging Maximum Element Search Code (Comparison Direction Reversed in C)

The following C function attempts to find the maximum element in an array, but the

comparison condition is reversed, causing the variable meant for the maximum to move toward smaller values instead. Carefully analyze the update logic and explain why the function may return the minimum or an incorrect intermediate value. Apply systematic debugging so that the tracking variable is updated only when a larger element is encountered. Optimize the corrected implementation for clarity, robustness, and proper input handling. Analyze the time complexity of the corrected solution and rewrite the final C code with suitable comments.

```
int findMax(int arr[], int n) {
    int i, maxVal = arr[0];
    for (i = 1; i < n; i++) {
        if (arr[i] < maxVal) // ERROR
            maxVal = arr[i];
    }
    return maxVal;
}
```

Q110. Debugging Insertion Sort Code (Wrong Shift Source Index in C)

The following Insertion Sort implementation in C shifts the wrong source element during the inner loop, which duplicates values and prevents the key from being inserted into the correct sorted position. Carefully analyze the shifting statement and explain why the current larger element must be copied to the next position before the key is inserted. Apply debugging step-by-step so that the sorted prefix is maintained correctly after every outer-loop iteration. Optimize the corrected implementation for readability and consistent indexing. Analyze the corrected algorithm in terms of time complexity and rewrite the final clean C code with proper comments.

```
void insertionSort(int arr[], int n) {
    int i, j, key;
    for (i = 1; i < n; i++) {
        key = arr[i];
        j = i - 1;
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j + 1]; // ERROR
            j--;
        }
        arr[j + 1] = key;
    }
}
```

Q111. Debugging Selection Sort Code (Minimum Index Reused Across Passes in C)

The following Selection Sort implementation in C incorrectly preserves the previous pass's minimum index by initializing it only once outside the outer loop, which causes later passes to compare against an outdated position. Carefully analyze why the sorted prefix and unsorted region are no longer handled independently. Apply systematic debugging so that the minimum index is reset correctly at the beginning of every pass. Optimize the corrected implementation by avoiding unnecessary swaps when the current element is already in its correct place. Analyze the time complexity of the corrected version and rewrite the final optimized C code with proper comments.

```

void selectionSort(int arr[], int n) {
    int i, j, min_idx = 0, temp; // ERROR
    for (i = 0; i < n - 1; i++) {
        for (j = i + 1; j < n; j++) {
            if (arr[j] < arr[min_idx]) {
                min_idx = j;
            }
        }
        temp = arr[i];
        arr[i] = arr[min_idx];
        arr[min_idx] = temp;
    }
}

```

Q112. Debugging Bubble Sort Code (Incorrect Adjacent Swap Target in C)

The following Bubble Sort implementation in C compares adjacent elements correctly, but the swap writes one of the values into the wrong location, resulting in duplicated elements and loss of data. Carefully inspect the swapping logic and explain why correct exchange requires both adjacent positions to be updated consistently. Apply systematic debugging so that each inversion is resolved without corrupting the array contents. Optimize the corrected implementation using a swap flag for early termination when the array becomes sorted. Analyze the best, average, and worst-case complexity and rewrite the final C code with proper comments.

```

void bubbleSort(int arr[], int n) {
    int i, j, temp;
    for (i = 0; i < n - 1; i++) {
        for (j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = arr[j]; // ERROR
            }
        }
    }
}

```

Q113. Debugging Linear Search Code (Loop Starts from Wrong Position in C)

The following Linear Search implementation in C begins searching from index 1, which causes the first array element to be skipped even when it contains the target. Carefully analyze the loop initialization and explain why this breaks correctness for valid matches at the first position. Apply systematic debugging so that the search begins from the correct starting index and checks every element. Optimize the corrected implementation for simplicity and clear failure handling. Analyze the time complexity of the corrected solution and rewrite the final C code with suitable comments.

```

int linearSearch(int arr[], int n, int key) {
    int i;
    for (i = 1; i < n; i++) { // ERROR
        if (arr[i] == key)

```

```

        return i;
    }
    return -1;
}

```

Q114. Debugging Binary Search Code (Incorrect Midpoint Update Creates Infinite Loop in C)

The following Binary Search implementation in C updates the lower and upper bounds incorrectly by assigning them to `mid` rather than moving beyond the midpoint, which may cause the search interval not to shrink and lead to an infinite loop. Carefully inspect the boundary updates and explain why the midpoint must be excluded after comparison. Apply systematic debugging so that each step strictly reduces the search space while preserving all valid candidates. Optimize the corrected implementation for clear structure and edge-case reliability. Analyze the best, average, and worst-case complexity and rewrite the final C code with proper documentation.

```

int binarySearch(int arr[], int n, int key) {
    int low = 0, high = n - 1, mid;
    while (low <= high) {
        mid = (low + high) / 2;
        if (arr[mid] == key)
            return mid;
        else if (arr[mid] < key)
            low = mid; // ERROR
        else
            high = mid; // ERROR
    }
    return -1;
}

```

Q115. Debugging Closest Pair Code (Incorrect Distance Formula Uses Mixed Coordinates in C)

The following brute force Closest Pair implementation in C mixes x-coordinates and y-coordinates from different points incorrectly while computing the Euclidean distance, which produces meaningless geometric results. Carefully analyze the formula and explain why each distance component must be computed using matching coordinate dimensions. Apply systematic debugging so that the x-difference and y-difference are calculated correctly for every valid pair. Optimize the implementation by storing point coordinates in temporary variables for readability. Analyze the time complexity of the corrected solution and rewrite the final C code with proper comments.

```

#include <math.h>
double closestPair(int x[], int y[], int n) {
    int i, j;
    double minDist = 1e9;
    for (i = 0; i < n; i++) {
        for (j = i + 1; j < n; j++) {
            double dist = sqrt((x[i] - y[j]) * (x[i] - y[j]) + // ERROR
                               (y[i] - x[j]) * (y[i] - x[j])); // ERROR

```

```

        if (dist < minDist)
            minDist = dist;
    }
}
return minDist;
}

```

Q116. Debugging Convex Hull Orientation Logic (Endpoints Not Excluded from Side Test in C)

The following Convex Hull helper in C checks all points, including the two endpoints of the candidate edge, while deciding whether the edge belongs to the hull. Carefully analyze why the endpoints should not affect the side-consistency test and how their inclusion can distort the logic in some implementations. Apply systematic debugging so that only the remaining points are used to validate the candidate edge. Optimize the corrected implementation for clearer structure and reduced unnecessary evaluations. Analyze the time complexity of the corrected brute force solution and rewrite the final C logic with suitable comments.

```

int isHullEdge(int p1, int p2, int x[], int y[], int n) {
    int i;
    for (i = 0; i < n; i++) {
        int val = (y[p2] - y[p1]) * (x[i] - x[p2]) - (x[p2] - x[p1]) * (y[i] - y[p2]);
        if (val < 0)
            return 0;
    }
    return 1;
}

```

Q117. Debugging String Matching Code (Premature Match Printing Inside Inner Loop in C)

The following brute force string matching implementation in C prints a match position from inside the inner loop before all characters of the pattern have been verified, causing false positives whenever only the first few characters match. Carefully inspect the control flow and explain why match reporting must happen only after the entire pattern comparison succeeds. Apply systematic debugging so that a position is printed only after every character matches consecutively. Optimize the corrected implementation for clarity and maintainable success detection. Analyze the time complexity of the corrected version and rewrite the final C code with proper comments.

```

#include <stdio.h>
#include <string.h>
void stringMatch(char text[], char pattern[]) {
    int n = strlen(text), m = strlen(pattern);
    int i, j;
    for (i = 0; i <= n - m; i++) {
        for (j = 0; j < m; j++) {
            if (text[i + j] != pattern[j])
                break;
            printf("Match at %d\n", i); // ERROR
        }
    }
}

```

```

    }
}

```

Q118. Debugging Pair Sum Code (Self-Pair Allowed Through Wrong Inner Loop Start in C)

The following brute force Pair Sum implementation in C starts the inner loop from the same index as the outer loop, which allows an element to be paired with itself and may produce invalid results when the target equals twice a single value. Carefully analyze why only distinct indices should be considered in this task. Apply systematic debugging so that each valid unordered pair is checked exactly once without self-comparison. Optimize the corrected implementation for readability and reduced redundancy. Analyze the time complexity of the corrected solution and rewrite the final C code with proper comments.

```

int pairSum(int arr[], int n, int target, int *a, int *b) {
    int i, j;
    for (i = 0; i < n; i++) {
        for (j = i; j < n; j++) { // ERROR
            if (arr[i] + arr[j] == target) {
                *a = i;
                *b = j;
                return 1;
            }
        }
    }
    return 0;
}

```

Q119. Debugging Maximum Element Search Code (Unsafe Access for Empty Array in C)

The following maximum-element function in C directly initializes the maximum with `arr[0]` without checking whether the array is empty, which leads to undefined behavior for invalid input sizes. Carefully analyze why robust code must validate input before accessing the first element. Apply systematic debugging so that the function safely handles empty arrays and then performs a correct scan for the largest value. Optimize the corrected implementation for defensive programming and clear error signaling. Analyze the time complexity of the corrected solution and rewrite the final C code with suitable comments.

```

int findMax(int arr[], int n) {
    int i, maxVal = arr[0]; // ISSUE
    for (i = 1; i < n; i++) {
        if (arr[i] > maxVal)
            maxVal = arr[i];
    }
    return maxVal;
}

```

Q120. Debugging Insertion Sort Code (Wrong Key Assignment Before Shift in C)

The following Insertion Sort implementation in C assigns the key from the wrong

array position before the shifting process begins, causing the algorithm to preserve an incorrect value and produce invalid sorted output. Carefully analyze how the key should represent the current element being inserted into the sorted prefix. Apply systematic debugging so that the correct key is stored first, larger elements are shifted right, and the saved key is inserted into its proper position. Optimize the corrected implementation for readability and consistent indexing. Analyze the corrected algorithm in terms of time complexity and rewrite the final clean C code with proper comments.

```
void insertionSort(int arr[], int n) {
    int i, j, key;
    for (i = 1; i < n; i++) {
        key = arr[i - 1]; // ERROR
        j = i - 1;
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}
```

Q121. Debugging Selection Sort Code (Wrong Minimum Comparison Range in C)

The following Selection Sort implementation in C incorrectly limits the inner loop to $n - 1$, which causes the final element of the unsorted region to remain unchecked during minimum selection. Carefully analyze why skipping the last candidate can leave a smaller element in the wrong place. Apply systematic debugging so that every element in the unsorted portion is examined before deciding the minimum index. Optimize the corrected implementation by avoiding unnecessary swaps when the selected minimum is already at the current position. Analyze the time complexity of the corrected version and rewrite the final optimized C code with proper comments.

```
void selectionSort(int arr[], int n) {
    int i, j, min_idx, temp;
    for (i = 0; i < n - 1; i++) {
        min_idx = i;
        for (j = i + 1; j < n - 1; j++) { // ERROR
            if (arr[j] < arr[min_idx]) {
                min_idx = j;
            }
        }
        temp = arr[i];
        arr[i] = arr[min_idx];
        arr[min_idx] = temp;
    }
}
```

Q122. Debugging Bubble Sort Code (Outer Pass Count Reduced Incorrectly in C)

The following Bubble Sort implementation in C performs one fewer outer pass than required, which may leave the array partially unsorted for certain worst-case inputs. Carefully inspect the outer loop bound and explain why all necessary passes are not guaranteed. Apply systematic debugging so that the algorithm performs enough passes to move every element to its correct position. Optimize the corrected implementation by adding a swap flag to terminate early when the array becomes sorted. Analyze the best, average, and worst-case complexity and rewrite the final C code with proper comments.

```
void bubbleSort(int arr[], int n) {
    int i, j, temp;
    for (i = 0; i < n - 2; i++) { // ERROR
        for (j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}
```

Q123. Debugging Linear Search Code (Incorrect Success Condition in C)

The following Linear Search implementation in C checks the wrong condition and returns success when the current element is different from the key instead of equal to it. Carefully analyze the comparison logic and explain why the search fails almost immediately for valid inputs. Apply systematic debugging so that the function returns the index only when the key is actually found. Optimize the corrected implementation for clear and maintainable control flow. Analyze the time complexity of the corrected solution and rewrite the final C code with suitable comments.

```
int linearSearch(int arr[], int n, int key) {
    int i;
    for (i = 0; i < n; i++) {
        if (arr[i] != key) // ERROR
            return i;
    }
    return -1;
}
```

Q124. Debugging Binary Search Code (Midpoint Computation Uses Incorrect Formula in C)

The following Binary Search implementation in C computes the midpoint using subtraction in the wrong direction, which can produce negative indices or an invalid search interval. Carefully inspect the midpoint formula and explain why the midpoint must always lie between the current lower and upper bounds. Apply systematic debugging so that the midpoint is computed safely and consistently at every iteration. Optimize the corrected implementation for robustness and clear boundary handling. Analyze the best, average, and worst-case complexity and rewrite the final C code with proper documentation.

```

int binarySearch(int arr[], int n, int key) {
    int low = 0, high = n - 1, mid;
    while (low <= high) {
        mid = low + (low - high) / 2; // ERROR
        if (arr[mid] == key)
            return mid;
        else if (arr[mid] < key)
            low = mid + 1;
        else
            high = mid - 1;
    }
    return -1;
}

```

Q125. Debugging Closest Pair Code (Pair Comparison Starts from Wrong Index in C)

The following brute force Closest Pair implementation in C starts the inner loop from index 0 for every outer-loop point, which rechecks old pairs, includes self-comparisons, and may incorrectly return zero as the minimum distance. Carefully analyze why each pair of distinct points should be considered exactly once. Apply systematic debugging so that only valid pairs with $j > i$ are evaluated. Optimize the corrected implementation to reduce redundant calculations while preserving brute force behavior. Analyze the time complexity of the corrected solution and rewrite the final C code with proper comments.

```

#include <math.h>
double closestPair(int x[], int y[], int n) {
    int i, j;
    double minDist = 1e9;
    for (i = 0; i < n; i++) {
        for (j = 0; j < n; j++) { // ERROR
            double dist = sqrt((x[i] - x[j]) * (x[i] - x[j]) +
                               (y[i] - y[j]) * (y[i] - y[j]));
            if (dist < minDist)
                minDist = dist;
        }
    }
    return minDist;
}

```

Q126. Debugging Convex Hull Orientation Logic (Positive and Negative Sides Not Distinguished in C)

The following brute force Convex Hull helper in C uses the same flag update for both positive and negative orientation values, which prevents it from correctly determining whether points lie on both sides of a candidate edge. Carefully analyze why hull validation depends on tracking both sides independently. Apply systematic debugging so that positive and negative orientations are recorded separately and mixed-side edges are rejected correctly. Optimize the corrected implementation for clear geometric reasoning and reduced unnecessary checks. Analyze the time

complexity of the corrected brute force solution and rewrite the final C logic with suitable comments.

```
int isHullEdge(int p1, int p2, int x[], int y[], int n) {
    int i, pos = 0, neg = 0;
    for (i = 0; i < n; i++) {
        int val = (y[p2] - y[p1]) * (x[i] - x[p2]) - (x[p2] - x[p1]) * (y[i] - y[p2]);
        if (val > 0)
            pos = 1;
        if (val < 0) // ERROR
            neg = 1;
    }
    return !(pos && neg);
}
```

Q127. Debugging String Matching Code (Inner Loop Starts with Wrong Text Offset in C)

The following brute force string matching implementation in C compares the pattern using `text[j]` instead of `text[i + j]`, which ignores the current alignment and repeatedly checks only the beginning of the text. Carefully analyze why pattern matching must shift the comparison window for each outer-loop position. Apply systematic debugging so that the pattern is compared against the correct substring of the text at every alignment. Optimize the corrected implementation for clarity while preserving the brute force strategy. Analyze the time complexity of the corrected version and rewrite the final C code with proper comments.

```
#include <stdio.h>
#include <string.h>
void stringMatch(char text[], char pattern[]) {
    int n = strlen(text), m = strlen(pattern);
    int i, j;
    for (i = 0; i <= n - m; i++) {
        for (j = 0; j < m; j++) {
            if (text[j] != pattern[j]) // ERROR
                break;
        }
        if (j == m)
            printf("Match at %d\n", i);
    }
}
```

Q128. Debugging Pair Sum Code (Wrong Output Parameters Assigned in C)

The following brute force Pair Sum implementation in C correctly identifies a valid pair, but it assigns the output indices in reversed variables and may confuse the caller about which positions were found. Carefully inspect the output assignment logic and explain why consistency in reporting indices is important. Apply systematic debugging so that the function stores the correct pair positions in the intended output parameters. Optimize the corrected implementation to ensure each unordered pair is checked only once. Analyze the time complexity of the corrected solution and rewrite the final C code with proper comments.

```

int pairSum(int arr[], int n, int target, int *a, int *b) {
    int i, j;
    for (i = 0; i < n; i++) {
        for (j = i + 1; j < n; j++) {
            if (arr[i] + arr[j] == target) {
                *b = i; // ERROR
                *a = j; // ERROR
                return 1;
            }
        }
    }
    return 0;
}

```

Q129. Debugging Maximum Element Search Code (Loop Omits Final Element in C)

The following maximum-element function in C stops its scan before reaching the last element of the array, which may cause the true maximum to be missed when it appears at the end. Carefully analyze the loop bound and explain why all array elements must be examined in a complete linear scan. Apply systematic debugging so that the final element is included in the comparison process. Optimize the corrected implementation for robustness and clear input validation. Analyze the time complexity of the corrected solution and rewrite the final C code with suitable comments.

```

int findMax(int arr[], int n) {
    int i, maxVal = arr[0];
    for (i = 1; i < n - 1; i++) { // ERROR
        if (arr[i] > maxVal)
            maxVal = arr[i];
    }
    return maxVal;
}

```

Q130. Debugging Insertion Sort Code (Wrong While Condition Uses Inverted Order in C)

The following Insertion Sort implementation in C uses the wrong comparison direction in the while-loop, causing it to shift smaller elements instead of larger ones and produce descending-like behavior in parts of the array. Carefully analyze how the sorted prefix invariant is broken by the inverted condition. Apply systematic debugging so that only elements greater than the key are shifted to the right before insertion. Optimize the corrected implementation for readability and consistent indexing. Analyze the corrected algorithm in terms of time complexity and rewrite the final clean C code with proper comments.

```

void insertionSort(int arr[], int n) {
    int i, j, key;
    for (i = 1; i < n; i++) {
        key = arr[i];
        j = i - 1;
        while (j >= 0 && arr[j] < key) { // ERROR

```

```

        arr[j + 1] = arr[j];
        j--;
    }
    arr[j + 1] = key;
}
}

```

Q131. Debugging Selection Sort Code (Swap Performed with Wrong Index in C)

The following Selection Sort implementation in C correctly finds the minimum index in each pass, but the swap uses `j` after the inner loop finishes instead of `min_idx`, causing undefined behavior and incorrect output. Carefully analyze why the loop variable is no longer valid for swapping once the search for the minimum is complete. Apply systematic debugging so that the selected minimum element is exchanged with the current position using the correct index. Optimize the corrected implementation by skipping the swap when the minimum is already in place. Analyze the time complexity of the corrected version and rewrite the final optimized C code with proper comments.

```

void selectionSort(int arr[], int n) {
    int i, j, min_idx, temp;
    for (i = 0; i < n - 1; i++) {
        min_idx = i;
        for (j = i + 1; j < n; j++) {
            if (arr[j] < arr[min_idx]) {
                min_idx = j;
            }
        }
        temp = arr[i];
        arr[i] = arr[j]; // ERROR
        arr[j] = temp;   // ERROR
    }
}

```

Q132. Debugging Bubble Sort Code (Comparison Uses Reversed Order for Ascending Sort in C)

The following Bubble Sort implementation in C uses a reversed comparison condition, which pushes larger elements toward the beginning and produces descending order instead of ascending order. Carefully inspect the adjacent comparison logic and explain why the current condition violates the intended sorting direction. Apply systematic debugging so that neighboring elements are exchanged only when the left element is greater than the right element for ascending order. Optimize the corrected implementation using a swap flag for early termination. Analyze the best, average, and worst-case complexity and rewrite the final C code with proper comments.

```

void bubbleSort(int arr[], int n) {
    int i, j, temp;
    for (i = 0; i < n - 1; i++) {
        for (j = 0; j < n - i - 1; j++) {
            if (arr[j] < arr[j + 1]) { // ERROR
                temp = arr[j];

```

```

        arr[j] = arr[j + 1];
        arr[j + 1] = temp;
    }
}
}
}

```

Q133. Debugging Linear Search Code (Missing Full Failure Handling in C)

The following Linear Search implementation in C returns the correct index when the key is found, but it has no explicit failure return after the loop, which leads to undefined behavior when the element is absent. Carefully analyze why every non-void function must return a valid value on all execution paths. Apply systematic debugging so that the function correctly reports failure after checking the entire array. Optimize the corrected implementation for clarity and safe control flow. Analyze the time complexity of the corrected solution and rewrite the final C code with suitable comments.

```

int linearSearch(int arr[], int n, int key) {
    int i;
    for (i = 0; i < n; i++) {
        if (arr[i] == key)
            return i;
    }
    /* ERROR: missing return */
}

```

Q134. Debugging Binary Search Code (High Boundary Initialized Incorrectly in C)

The following Binary Search implementation in C initializes the upper boundary to **n** instead of **n - 1**, which may cause out-of-bounds midpoint access during execution. Carefully inspect the boundary setup and explain why valid array indices must remain within **0** to **n - 1**. Apply systematic debugging so that the search interval is initialized and maintained correctly for all input sizes. Optimize the corrected implementation for safe index handling and clear structure. Analyze the best, average, and worst-case complexity and rewrite the final C code with proper documentation.

```

int binarySearch(int arr[], int n, int key) {
    int low = 0, high = n, mid; // ERROR
    while (low <= high) {
        mid = (low + high) / 2;
        if (arr[mid] == key)
            return mid;
        else if (arr[mid] < key)
            low = mid + 1;
        else
            high = mid - 1;
    }
    return -1;
}

```

Q135. Debugging Closest Pair Code (Wrong Distance Update Variable in C)

The following brute force Closest Pair implementation in C computes the distance correctly, but it updates `minDist` using only the x-coordinate difference instead of the full Euclidean distance, which breaks the geometric meaning of the result. Carefully analyze why the complete distance expression must be preserved once a closer pair is found. Apply systematic debugging so that the minimum distance variable always stores the true pair distance. Optimize the corrected implementation by reducing repeated coordinate access for better readability. Analyze the time complexity of the corrected solution and rewrite the final C code with proper comments.

```
#include <math.h>
double closestPair(int x[], int y[], int n) {
    int i, j;
    double minDist = 1e9;
    for (i = 0; i < n; i++) {
        for (j = i + 1; j < n; j++) {
            double dx = x[i] - x[j];
            double dy = y[i] - y[j];
            double dist = sqrt(dx * dx + dy * dy);
            if (dist < minDist)
                minDist = dx; // ERROR
        }
    }
    return minDist;
}
```

Q136. Debugging Convex Hull Orientation Logic (Candidate Edge Accepted Without Mixed-Side Check in C)

The following Convex Hull helper in C returns success as long as it encounters any non-negative orientation, but this is not sufficient to prove that all remaining points lie on a single side of the candidate edge. Carefully analyze why a valid hull edge must ensure that points are not distributed on both sides of the line. Apply systematic debugging so that the logic rejects mixed-side cases and accepts only proper boundary edges. Optimize the corrected implementation for clearer geometric reasoning and fewer unnecessary checks. Analyze the time complexity of the corrected brute force solution and rewrite the final C logic with suitable comments.

```
int isHullEdge(int p1, int p2, int x[], int y[], int n) {
    int i;
    for (i = 0; i < n; i++) {
        int val = (y[p2] - y[p1]) * (x[i] - x[p2]) - (x[p2] - x[p1]) * (y[i] - y[p2]);
        if (val >= 0)
            return 1; // ERROR
    }
    return 0;
}
```

Q137. Debugging String Matching Code (Wrong Success Condition After Inner Loop in C)

The following brute force string matching implementation in C uses `j == m - 1` as the success condition after the inner loop, which can incorrectly report a match when the loop stops early on a mismatch near the end. Carefully inspect why the full pattern

length must be completed before confirming success. Apply systematic debugging so that a match is reported only when all **m** characters have been compared successfully. Optimize the corrected implementation for clarity and maintainable control flow. Analyze the time complexity of the corrected version and rewrite the final C code with proper comments.

```
#include <stdio.h>
#include <string.h>
void stringMatch(char text[], char pattern[]) {
    int n = strlen(text), m = strlen(pattern);
    int i, j;
    for (i = 0; i <= n - m; i++) {
        for (j = 0; j < m; j++) {
            if (text[i + j] != pattern[j])
                break;
        }
        if (j == m - 1) // ERROR
            printf("Match at %d\n", i);
    }
}
```

Q138. Debugging Pair Sum Code (Premature Failure Return in C)

The following brute force Pair Sum implementation in C returns failure immediately after checking the first outer-loop element, which prevents the remaining pairs from ever being examined. Carefully analyze the location of the failure return statement and explain why it must occur only after all pair combinations have been tested. Apply systematic debugging so that every valid unordered pair is considered exactly once before concluding no solution exists. Optimize the corrected implementation for clarity and reduced redundant comparisons. Analyze the time complexity of the corrected solution and rewrite the final C code with proper comments.

```
int pairSum(int arr[], int n, int target, int *a, int *b) {
    int i, j;
    for (i = 0; i < n; i++) {
        for (j = i + 1; j < n; j++) {
            if (arr[i] + arr[j] == target) {
                *a = i;
                *b = j;
                return 1;
            }
        }
        return 0; // ERROR
    }
    return 0;
}
```

Q139. Debugging Maximum Element Search Code (Incorrect Handling of Negative-Only Arrays in C)

The following maximum-element function in C initializes the answer to **-1**, which produces incorrect output when all values in the array are smaller than **-1**. Carefully analyze why the initialization must depend on actual data values rather than an

assumed threshold. Apply systematic debugging so that the maximum element is initialized safely from the array after validating input size. Optimize the corrected implementation for robustness and clear error handling. Analyze the time complexity of the corrected solution and rewrite the final C code with suitable comments.

```
int findMax(int arr[], int n) {
    int i, maxVal = -1; // ERROR
    for (i = 0; i < n; i++) {
        if (arr[i] > maxVal)
            maxVal = arr[i];
    }
    return maxVal;
}
```

Q140. Debugging Insertion Sort Code (Inner Loop Shifts in Wrong Direction in C)

The following Insertion Sort implementation in C decreases the wrong array location during the shifting phase by copying from `arr[j + 1]` into `arr[j]`, which destroys values and breaks the sorted-prefix invariant. Carefully analyze why larger elements must move one position to the right, not to the left, before the key is inserted. Apply systematic debugging so that the shifting and insertion steps preserve all array elements correctly. Optimize the corrected implementation for readability and consistent indexing. Analyze the corrected algorithm in terms of time complexity and rewrite the final clean C code with proper comments.

```
void insertionSort(int arr[], int n) {
    int i, j, key;
    for (i = 1; i < n; i++) {
        key = arr[i];
        j = i - 1;
        while (j >= 0 && arr[j] > key) {
            arr[j] = arr[j + 1]; // ERROR
            j--;
        }
        arr[j + 1] = key;
    }
}
```

Q141. Debugging Selection Sort Code (Minimum Index Compared Against Wrong Loop Variable in C)

The following Selection Sort implementation in C compares the candidate element against `arr[j]` instead of `arr[min_idx]`, which prevents the algorithm from preserving the true minimum seen so far within the current pass. Carefully analyze why the minimum candidate must be updated relative to the current best position, not the current scanning position. Apply systematic debugging so that the smallest element of the unsorted region is tracked correctly. Optimize the corrected implementation by skipping unnecessary swaps when no movement is needed. Analyze the time complexity of the corrected version and rewrite the final optimized C code with proper comments.

```
void selectionSort(int arr[], int n) {
```

```

int i, j, min_idx, temp;
for (i = 0; i < n - 1; i++) {
    min_idx = i;
    for (j = i + 1; j < n; j++) {
        if (arr[j] < arr[min_idx]) { // ERROR
            min_idx = j;
        }
    }
    temp = arr[i];
    arr[i] = arr[min_idx];
    arr[min_idx] = temp;
}
}

```

Q142. Debugging Bubble Sort Code (Final Comparison of Each Pass Skipped in C)

The following Bubble Sort implementation in C stops the inner loop too early by using `< n - i - 2`, which causes the final adjacent comparison of each pass to be skipped. Carefully analyze why missing even one adjacent check can leave inversions unresolved and prevent complete sorting. Apply systematic debugging so that every required adjacent pair in the unsorted region is examined. Optimize the corrected implementation using a swap flag to reduce unnecessary passes when the array becomes sorted. Analyze the best, average, and worst-case complexity and rewrite the final C code with proper comments.

```

void bubbleSort(int arr[], int n) {
    int i, j, temp;
    for (i = 0; i < n - 1; i++) {
        for (j = 0; j < n - i - 2; j++) { // ERROR
            if (arr[j] > arr[j + 1]) {
                temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

```

Q143. Debugging Linear Search Code (Returned Value Is Element Instead of Index in C)

The following Linear Search implementation in C returns the element value when a match is found instead of returning its array index, which violates the intended specification of the function. Carefully analyze why the caller cannot identify the position of the key from this result when duplicates exist. Apply systematic debugging so that the function returns the correct index for the first matching element. Optimize the corrected implementation for simple and maintainable behavior. Analyze the time complexity of the corrected solution and rewrite the final C code with suitable comments.

```

int linearSearch(int arr[], int n, int key) {
    int i;

```

```

for (i = 0; i < n; i++) {
    if (arr[i] == key)
        return arr[i]; // ERROR
}
return -1;
}

```

Q144. Debugging Binary Search Code (Unsorted Input Assumption Not Checked in C)

The following Binary Search implementation in C assumes that the input array is sorted, but it is being called on arbitrary arrays without validation or preparation, which can produce meaningless results. Carefully analyze why Binary Search depends completely on ordered data and how this assumption affects correctness. Apply debugging by redesigning the usage logic so that either the input is guaranteed sorted or an appropriate preprocessing step is performed before search. Optimize the corrected approach by clearly documenting the precondition and handling edge cases safely. Analyze the best, average, and worst-case complexity of the corrected searching strategy and rewrite the final C code with proper comments.

```

int binarySearch(int arr[], int n, int key) {
    int low = 0, high = n - 1, mid;
    while (low <= high) {
        mid = (low + high) / 2;
        if (arr[mid] == key)
            return mid;
        else if (arr[mid] < key)
            low = mid + 1;
        else
            high = mid - 1;
    }
    return -1;
}
/* Called on unsorted arrays */

```

Q145. Debugging Closest Pair Code (Minimum Distance Returned Uninitialized for Small Input in C)

The following brute force Closest Pair implementation in C does not handle the case where the number of points is less than two, so the minimum-distance variable may be returned without a meaningful pair comparison. Carefully analyze why small-input validation is required before entering pairwise computation. Apply systematic debugging so that the function safely handles invalid or insufficient input sizes and returns an appropriate indicator. Optimize the corrected implementation for robustness and clear documentation of assumptions. Analyze the time complexity of the corrected solution and rewrite the final C code with proper comments.

```

#include <math.h>
double closestPair(int x[], int y[], int n) {
    int i, j;
    double minDist = 1e9;
    for (i = 0; i < n; i++) {
        for (j = i + 1; j < n; j++) {

```

```

        double dist = sqrt((x[i] - x[j]) * (x[i] - x[j]) +
                           (y[i] - y[j]) * (y[i] - y[j]));
        if (dist < minDist)
            minDist = dist;
    }
}
return minDist; /* ISSUE when n < 2 */
}

```

Q146. Debugging Convex Hull Orientation Logic (All Collinear Sets Mishandled in C)

The following brute force Convex Hull helper in C fails when all points are collinear because it assumes that a valid hull edge must strictly separate points to one side, which is not true for degenerate hulls. Carefully analyze why collinear datasets require special handling and why multiple outer segments may still be part of the hull representation. Apply systematic debugging so that the algorithm can correctly reason about degenerate or fully collinear point sets. Optimize the corrected implementation for clearer case separation and reliable behavior across edge cases. Analyze the time complexity of the corrected brute force solution and rewrite the final C logic with suitable comments.

```

int isHullEdge(int p1, int p2, int x[], int y[], int n) {
    int i, sign = 0;
    for (i = 0; i < n; i++) {
        int val = (y[p2] - y[p1]) * (x[i] - x[p2]) - (x[p2] - x[p1]) * (y[i] - y[p2]);
        if (val != 0 && sign == 0)
            sign = (val > 0) ? 1 : -1;
        else if ((val > 0 && sign < 0) || (val < 0 && sign > 0))
            return 0;
    }
    return 1; /* ISSUE on fully collinear sets */
}

```

Q147. Debugging String Matching Code (Pattern Longer Than Text Not Handled in C)

The following brute force string matching implementation in C assumes that the pattern length is less than or equal to the text length, but it does not explicitly handle the case where the pattern is longer. Carefully analyze why this edge case must be checked before beginning the search. Apply systematic debugging so that the function safely exits or reports no match when the pattern cannot fit within the text. Optimize the corrected implementation for robust control flow and clear readability. Analyze the time complexity of the corrected version and rewrite the final C code with proper comments.

```

#include <stdio.h>
#include <string.h>
void stringMatch(char text[], char pattern[]) {
    int n = strlen(text), m = strlen(pattern);
    int i, j;
    for (i = 0; i <= n - m; i++) {
        for (j = 0; j < m; j++) {

```

```

        if (text[i + j] != pattern[j])
            break;
    }
    if (j == m)
        printf("Match at %d\n", i);
}
}

```

Q148. Debugging Pair Sum Code (Output Variables Not Validated in C)

The following brute force Pair Sum implementation in C assumes that the output pointers `a` and `b` are always valid, which can cause runtime errors if null pointers are passed by the caller. Carefully analyze why pointer validation is important in C before writing results through indirection. Apply systematic debugging so that the function safely checks its output parameters and reports failure appropriately when they are invalid. Optimize the corrected implementation for defensive programming and clear API behavior. Analyze the time complexity of the corrected solution and rewrite the final C code with proper comments.

```

int pairSum(int arr[], int n, int target, int *a, int *b) {
    int i, j;
    for (i = 0; i < n; i++) {
        for (j = i + 1; j < n; j++) {
            if (arr[i] + arr[j] == target) {
                *a = i; /* ISSUE if a == NULL */
                *b = j; /* ISSUE if b == NULL */
                return 1;
            }
        }
    }
    return 0;
}

```

Q149. Debugging Maximum Element Search Code (Array Bounds Not Validated Before Access in C)

The following maximum-element search function in C accesses `arr[0]` without first checking whether the pointer is valid or whether the size is positive, which can lead to undefined behavior. Carefully analyze why low-level functions in C must validate both pointer and size assumptions before reading memory. Apply systematic debugging so that invalid input is handled safely and the maximum search proceeds only when the array is usable. Optimize the corrected implementation for defensive programming and clear return conventions. Analyze the time complexity of the corrected solution and rewrite the final C code with suitable comments.

```

int findMax(int arr[], int n) {
    int i, maxVal = arr[0]; /* ISSUE */
    for (i = 1; i < n; i++) {
        if (arr[i] > maxVal)
            maxVal = arr[i];
    }
    return maxVal;
}

```

Q150. Debugging Insertion Sort Code (Outer Loop Starts from Wrong Index in C)

The following Insertion Sort implementation in C starts the outer loop from index 0, which makes the algorithm treat the first element as a key unnecessarily and can interfere with correct initialization of the sorted prefix. Carefully analyze why insertion sort should begin from the second element, assuming the first element is already a sorted prefix of length one. Apply systematic debugging so that the algorithm inserts each subsequent element into the correct position within the growing sorted portion. Optimize the corrected implementation for readability, correctness, and stable behavior. Analyze the corrected algorithm in terms of time complexity and rewrite the final clean C code with proper comments.

```
void insertionSort(int arr[], int n) {
    int i, j, key;
    for (i = 0; i < n; i++) { // ERROR
        key = arr[i];
        j = i - 1;
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}
```

RUBRICS FOR CALCULATION

DEBUGGING & OPTIMISATION TASK					
Criteria	Weightage	Level (Exemplary) 4	Level (Proficient) 3	Level (Developing) 2	Level (Beginning) 1
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates	Shows	Limited analysis;	No analytical

		strong logical reasoning and clear justification of solutions	reasonable analysis with some justification	weak reasoning	reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation